

数字系统设计

Digital System Design

-----验证与测试

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信工程研究所

Zhejiang University

Spring 2026 – Lec. #10

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室（可以钉钉、微信或邮件联系）.

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社,2016.
 - 王金明著,《数字系统设计与Verilog HDL》,电子工业出版社,第8版
 - 《数字系统设计-补充讲义》,浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition,电子工业出版社,2005.
 - M.M.Mano, 数字设计(第四版),电子工业出版社,2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**

- 期中考试 **10%**
- Homework Sets **15% (12~20)%**
- Project **5% (5~7)%**
- Class Room Check+小测验等 **10% (7~10)%**
- 作业每周一上交, **截止期为课后一周内**有效。

- **Project**

- Project-1必做 (1 members)
- Project-2可选(成绩加分1%~5%, 但不超过平时成绩范围)

- Finial Exam期末**闭卷**考试 - **60%**

授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

讲义第九章 验证与测试

9.1 引言

- 系统可测性设计（design for testability, DFT）
 - DFT是整个设计过程一个非常重要的组成部分
 - 在设计流程中需要尽早考虑
 - 如果你不测试它，系统很可能不工作
- DFT策略
 - 提供必要的电路
 - » 以使测试过程快速且全面
 - 提供测试激励矢量
 - » 测试过程需要的
 - » 降低成本
 - 希望测试序列尽可能短
 - 但仍能覆盖大部分可能存在的缺陷。

9.2 生产测试过程

- 诊断测试

- 芯片和板级调试期间
- 识别和指出失效的部位
 - » 对于给定的失效部件

- 功能测试

- 确定一个制造出的元件是否能工作。
- 每一个制造出来的芯片都要经过这一测试
 - » 直接的影响芯片成本
 - » 测试尽可能简单快速

- 参数测试

- 在各种工作条件（如温度和电源电压）下
 - » 检查许多非离散参数，如噪声容限、传播延时和最大时钟频率
- 参数测试一般分为静态和动态测试。

9.2.生产测试过程

- **Introduction**

- The fabrication process is one of the most precise manufacturing methods we know of, but it is still not perfect.
- Each time a chip is made, there is a finite chance that at least one of the millions of transistors or wires will have an error in it.

- **Production test**

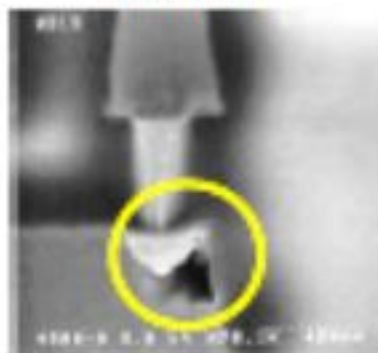
- Since users of the chips assume that the chip is function, we need to have some method to sort out the function chips from the bad ones.
- a critical part of manufacturin

- **Design for test.**

- Given the complexity of today's chips, designers need to add features to the chip to make production testing possible.

Manufacturing Process Isn't Perfect

Void under anchor



Silicon damage



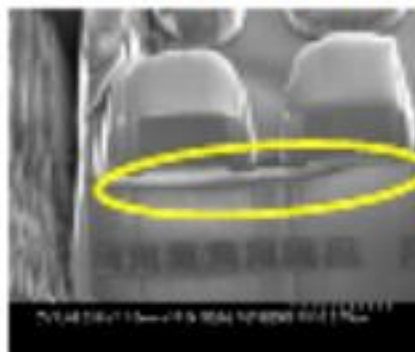
Metal2 extrusion/
ILD2 crack



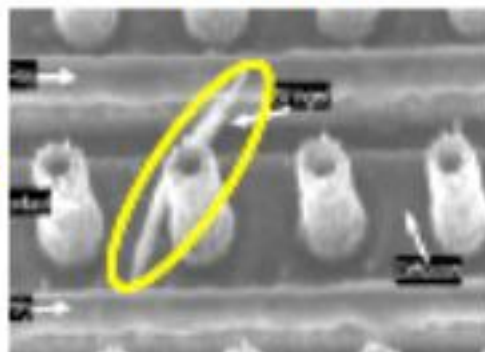
Metal 1 Shelving



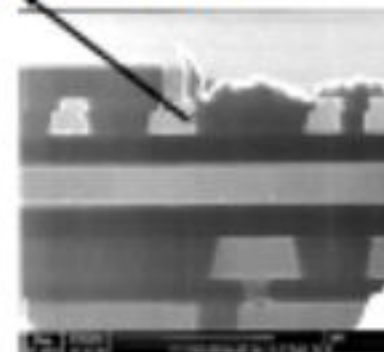
M4-M4 Short



Poly stringer



M4 Void Formations



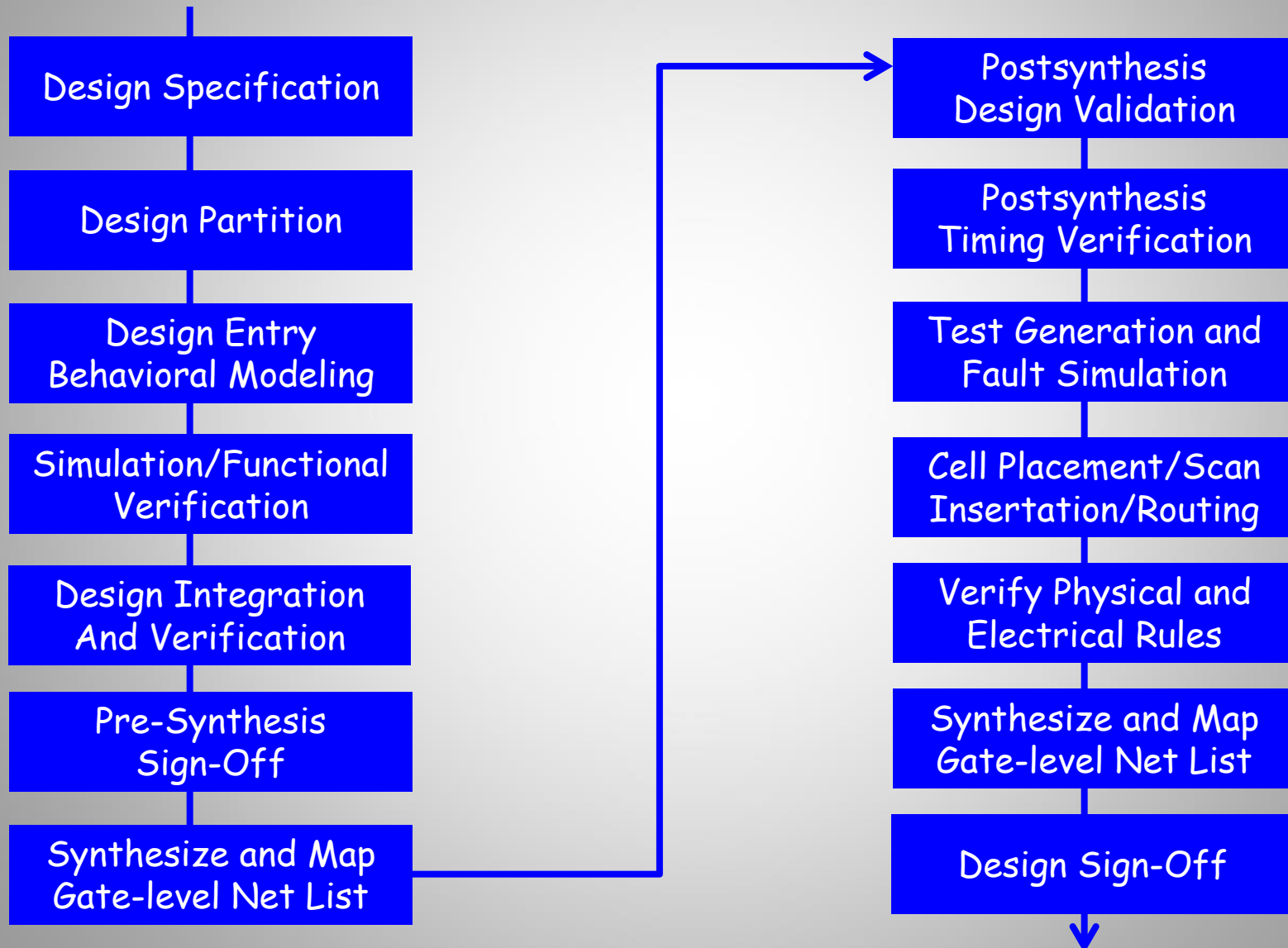
SEM

EE371

Source: [Spirakis ETW 2002]

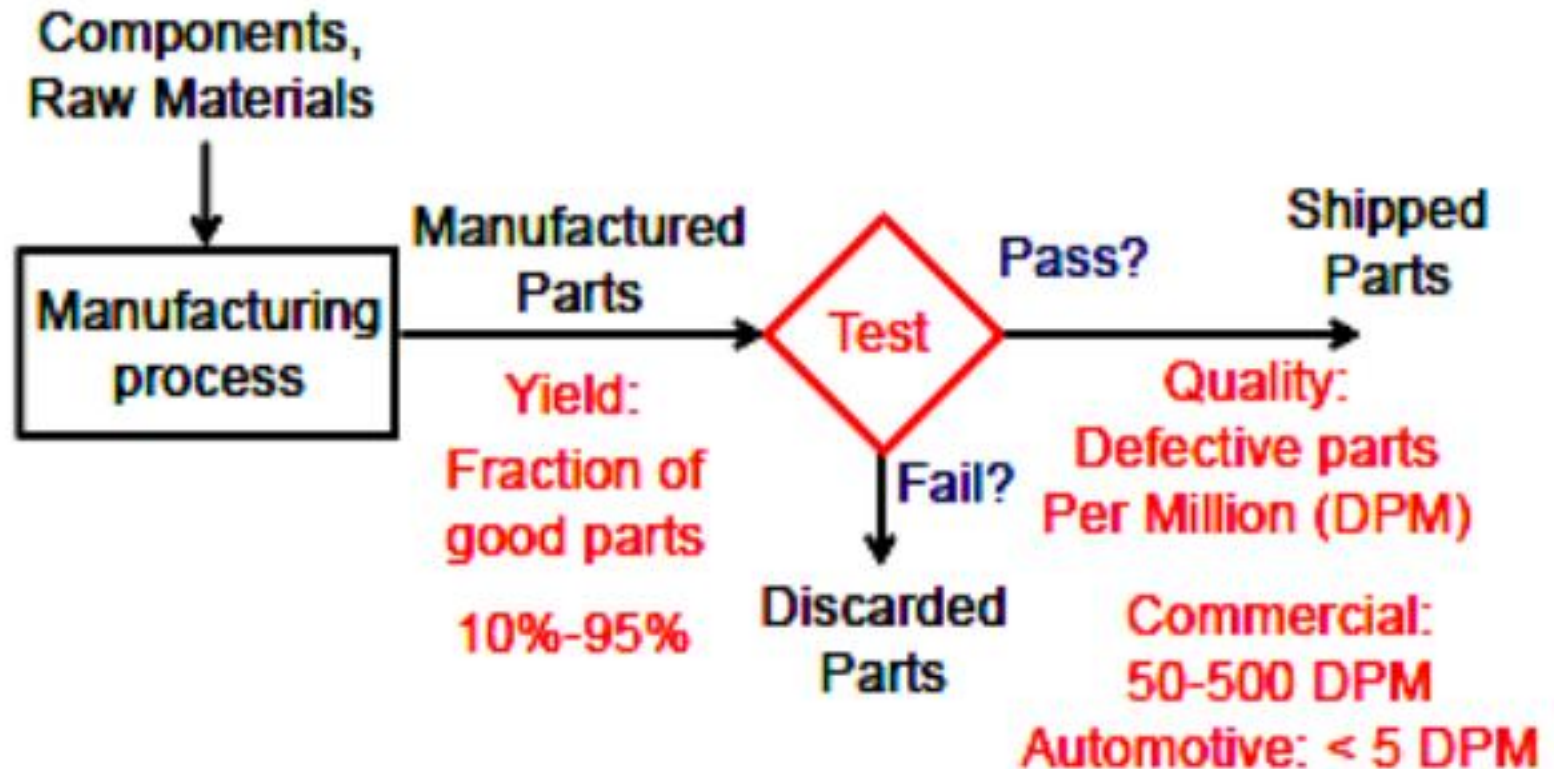
3

Design Methodology in Detail



一个典型的生产测试

Production Test

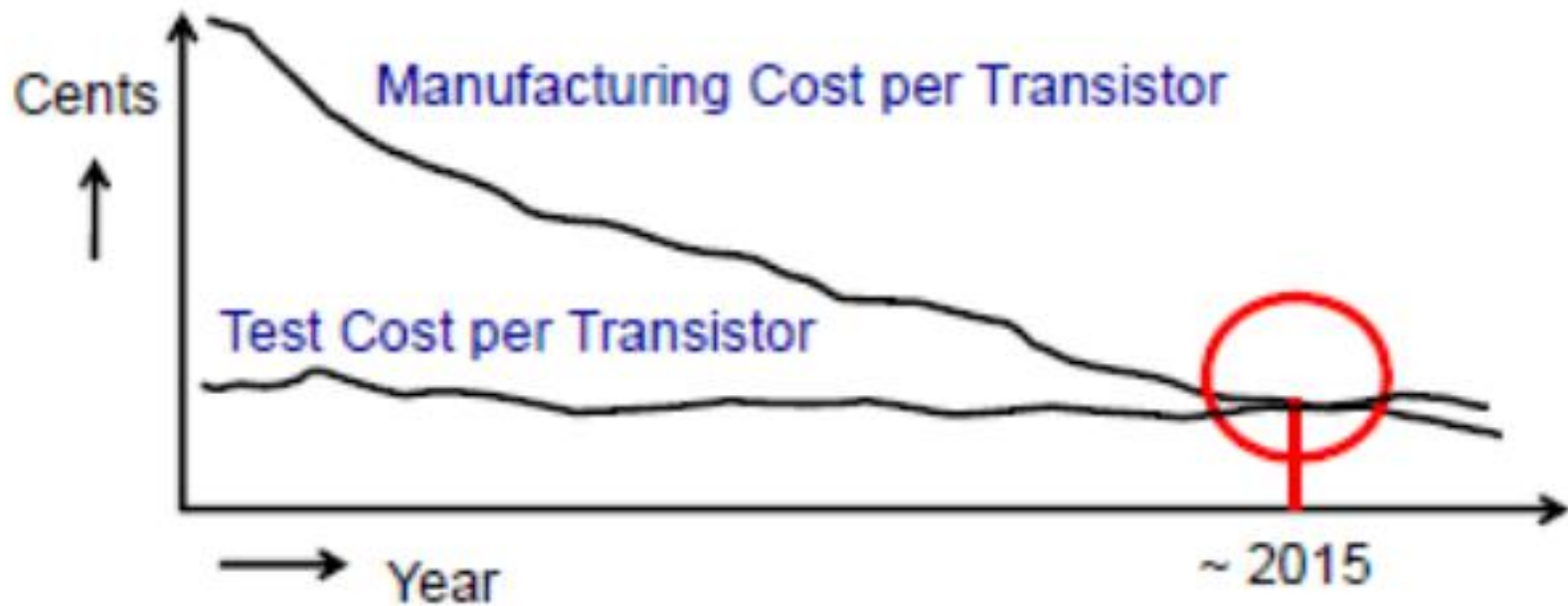


自动测试仪

Automatic Test Equipment (ATE)



Cost of Test



- [Gelsinger, IEEE Design & Test, Jan. 2000]
- ITRS roadmap

9.3.可测性设计

- Testing of Logic Circuits
 - Fault Models故障模型
 - Test Generation and Coverage
 - Fault Detection
 - Design for Test
- 在设计过程的早期考虑测试
 - 可能简化整个验证过程
 - 所有可能的输入矢量并观察相应的响应实现
 - » 验证该电路的正确性

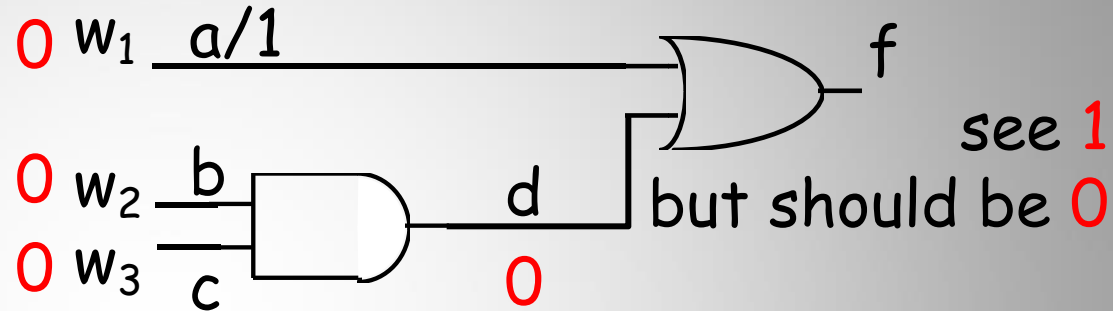
Fault Model故障模型

- **Stuck-At**固定故障**Model**

- Assume selected wires (gate input or output) are “stuck at”固定 logic value 0 or 1故障
- Models certain kinds of fabrication flaws that short circuit wires to ground or power, or broken wires that are floating
 - » Wire w stuck-at-0: $w/0$
 - » Wire w stuck-at-1: $w/1$
- Often assume there is only one fault at a time—even though in real circuits multiple simultaneous faults are possible and can mask each other
- Obviously a very simplistic model!

Fault Model

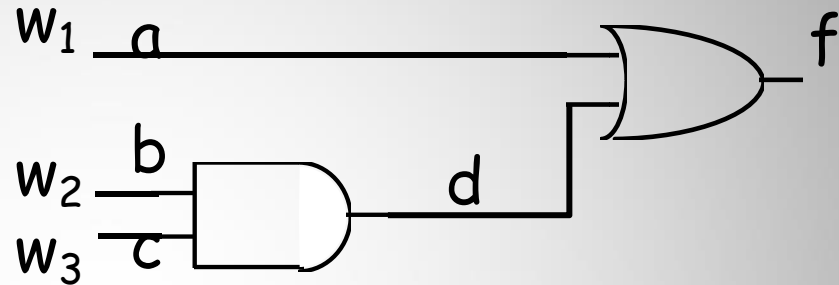
- Simple example:



- Generate a testcase to determine if a is stuck at 1
 - Try 000
 - If a stuck at 1, expect to see $f = 0$, but see 1 instead

Fault Model

- Simple example



Test w1 w2 w3	Fault Detected									
	a/0	a/1	b/0	b/1	c/0	c/1	d/0	d/1	f/0	f/1
000		X						X		X
001		X		X				X		X
010		X				X		X		X
011			X		X		X		X	
100	X								X	
101	X								X	
110	X								X	
111									X	

Test
Set

Problems with Fault Model故障模型

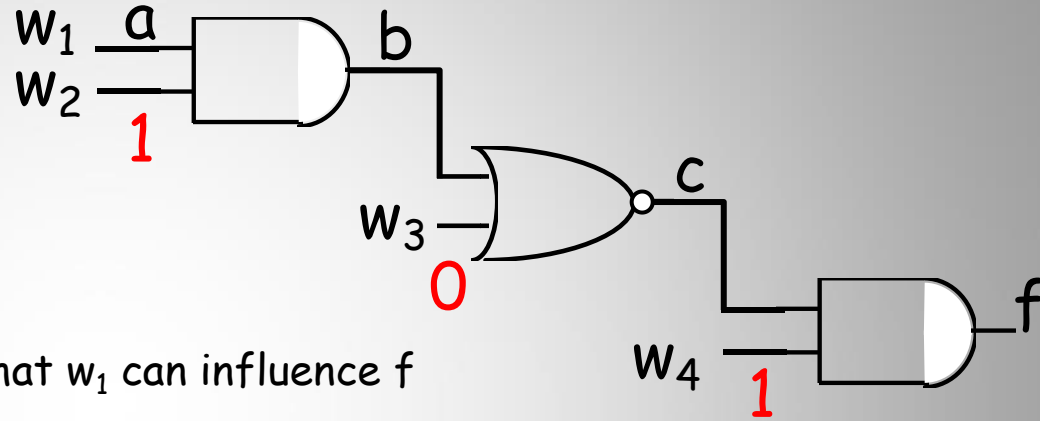
- In general, n -input circuits require much less than 2^n test inputs to cover all possible stuck-at-faults in the circuit
- However, this number is usually still too large in real circuits for practical purposes
- Finding minimum test cover is an **NP-hard problem** too (NP难题, non-deterministic polynomial缩写)

Path Sensitization

- Wire-at-time testing too laborious
- Better to focus on wiring paths, enabling multi-wire testing at the same time
- “Activate” a path so that changes in signal propagating along the path affects the output

Path Sensitization

- Simple Example:



To activate the path, set inputs so that w_1 can influence f

E.g., $w_2 = 1$, $w_3 = 0$, $w_4 = 1$

AND gates: one input at 1 passes the other input

NOR gates: one input at 0 inverts the other input

To test: w_1 set to 1 should generate $f = 0$ if path ok

faults $a/0$, $b/0$, $c/1$ cause $f = 1$

w_1 set to 0 should generate $f = 1$ if path ok

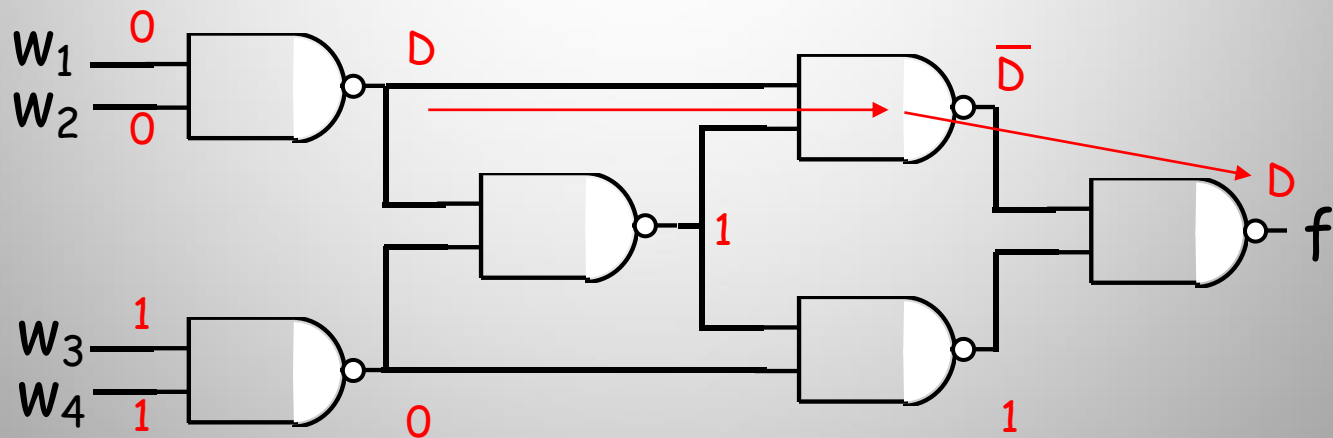
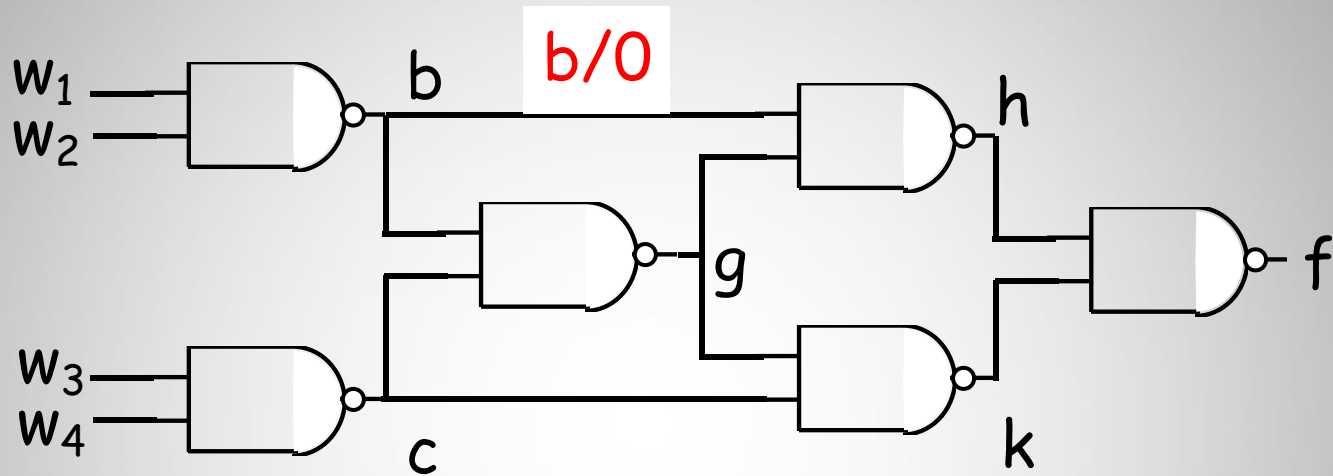
faults $a/1$, $b/1$, $c/0$ cause $f = 0$

One test can capture several faults at once!

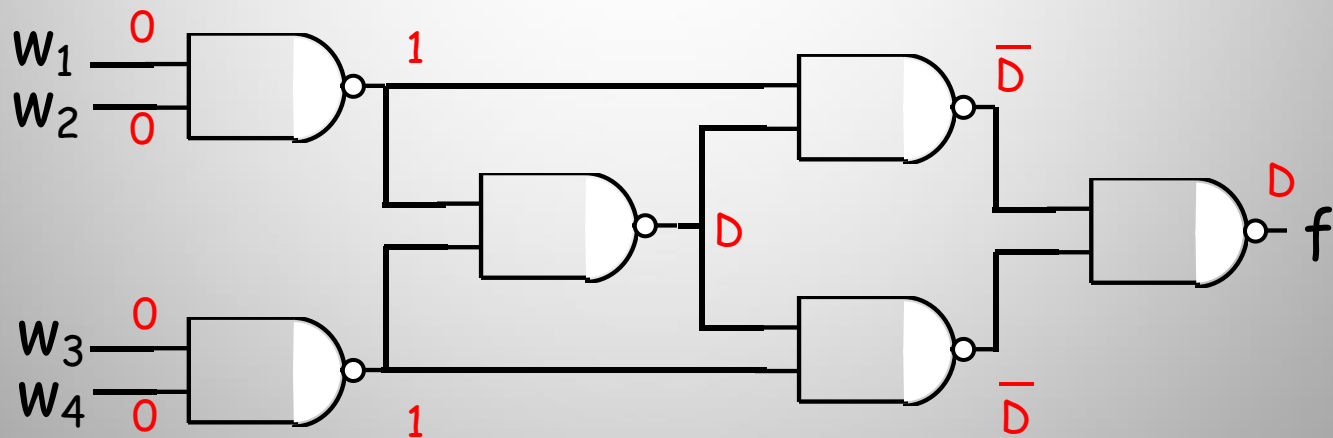
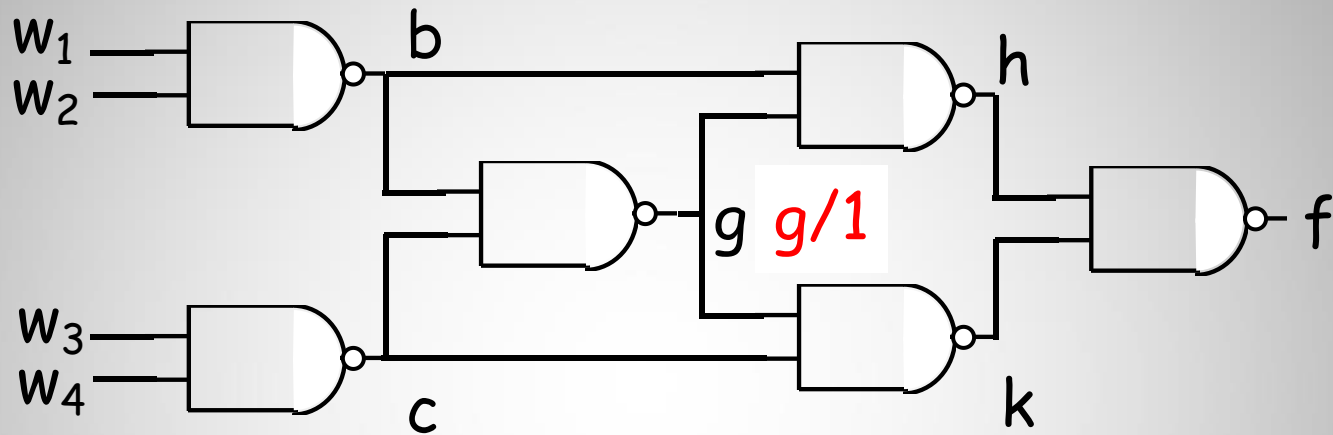
Path Sensitization

- **Good news:** one test checks for several faults
 - Number of paths much smaller than number of wires
 - Still an impractically large number of paths for large-scale circuits
- **Path idea** can be used to “propagate” a fault to the output to observe the fault
 - Set inputs and intermediate values so as to pass an internal wire to the output while setting inputs to drive that internal wire to a known value
 - If propagated value isn't as expected, then we have found a fault on the isolated wire

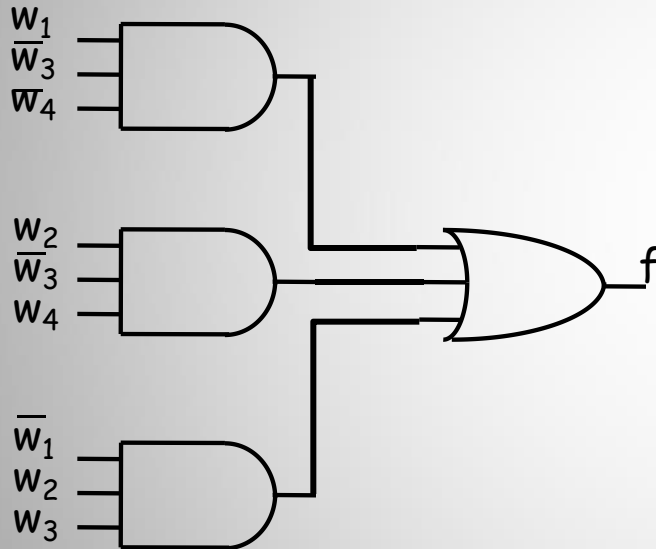
Fault Propagation



Fault Propagation

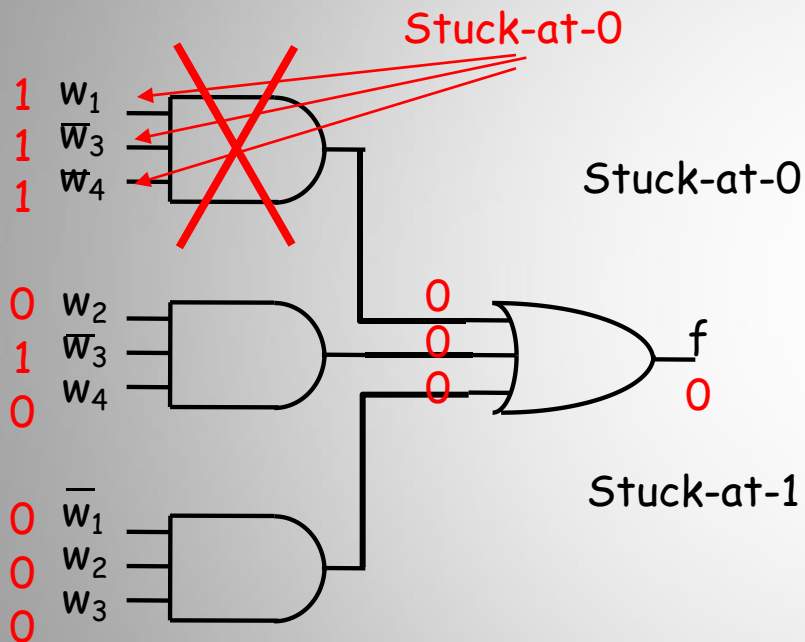


Tree Structured Circuits



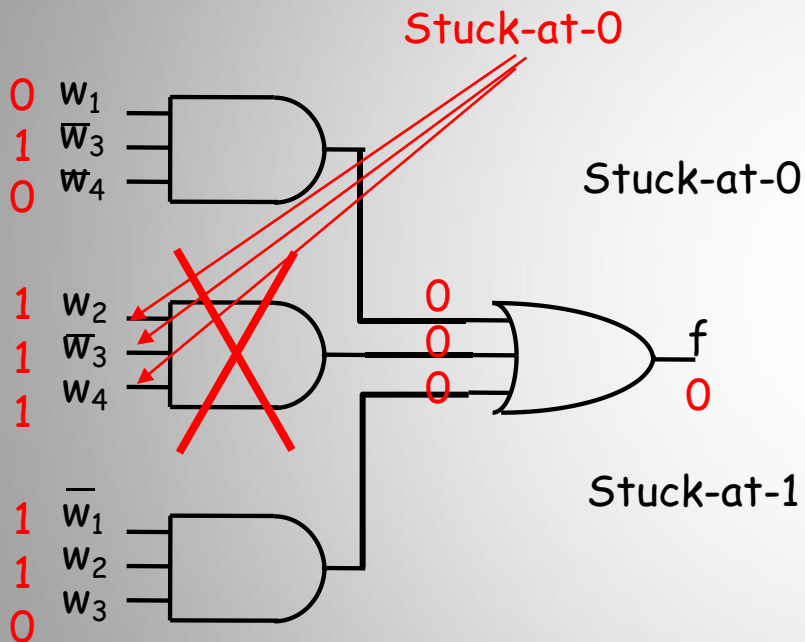
- To test inputs **stuck-at-0** at given AND gate
 - Set inputs at other gates to generate AND output of zero
 - Force inputs at selected gate to generate **a one**
 - If **f is 1** then circuit ok, else fault
- To test inputs **stuck-at-1** at given AND gate
 - Drive input to test to 0, rest of inputs driven to 1
 - Other gates driven with inputs that force gates to 0
 - If **f is 0** then OK, else fault

Tree Structured Circuits



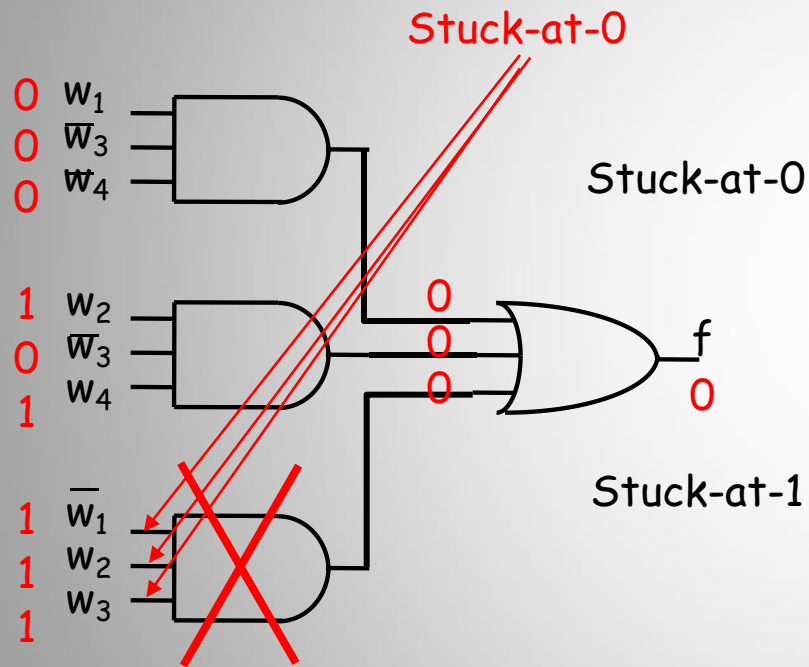
	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Tree Structured Circuits



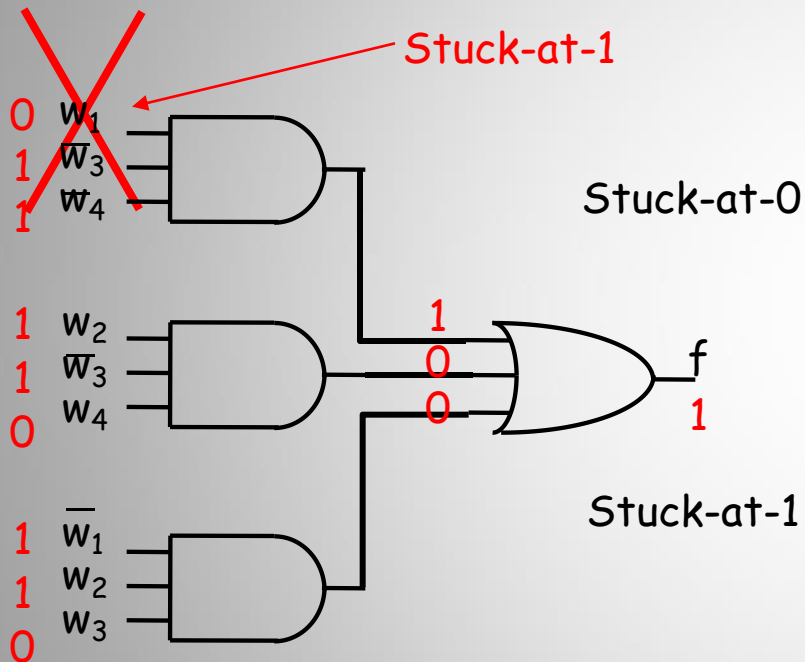
	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Tree Structured Circuits



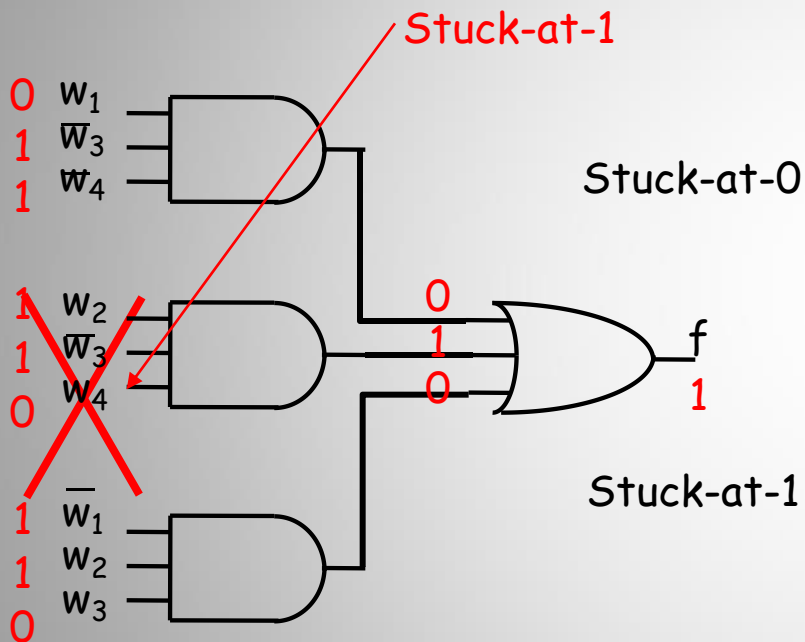
	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Tree Structured Circuits



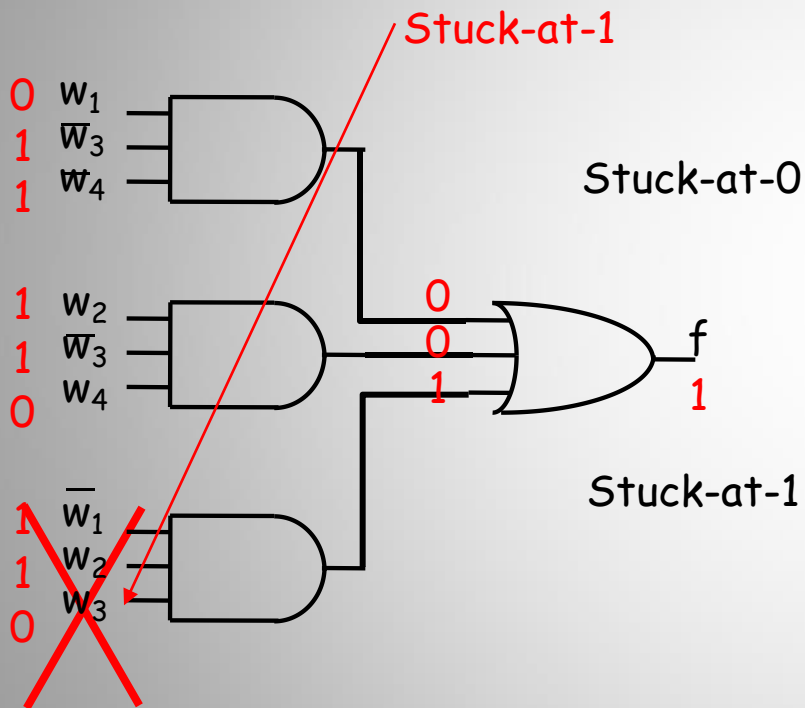
	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Tree Structured Circuits



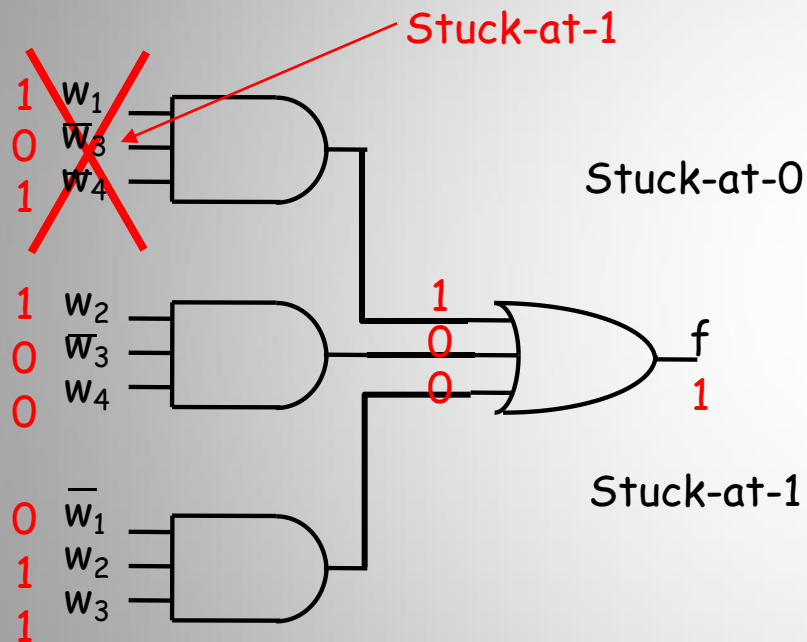
	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Tree Structured Circuits



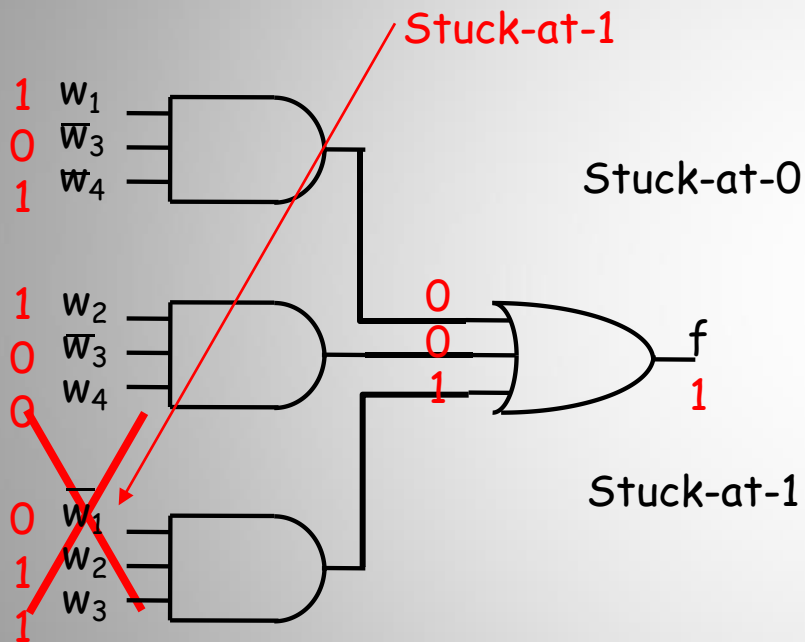
	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Tree Structured Circuits



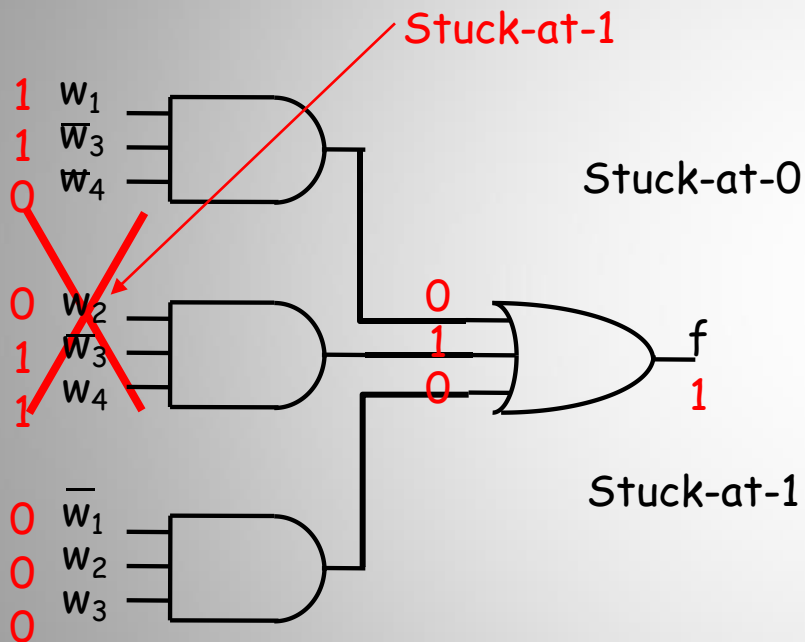
	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Tree Structured Circuits



	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

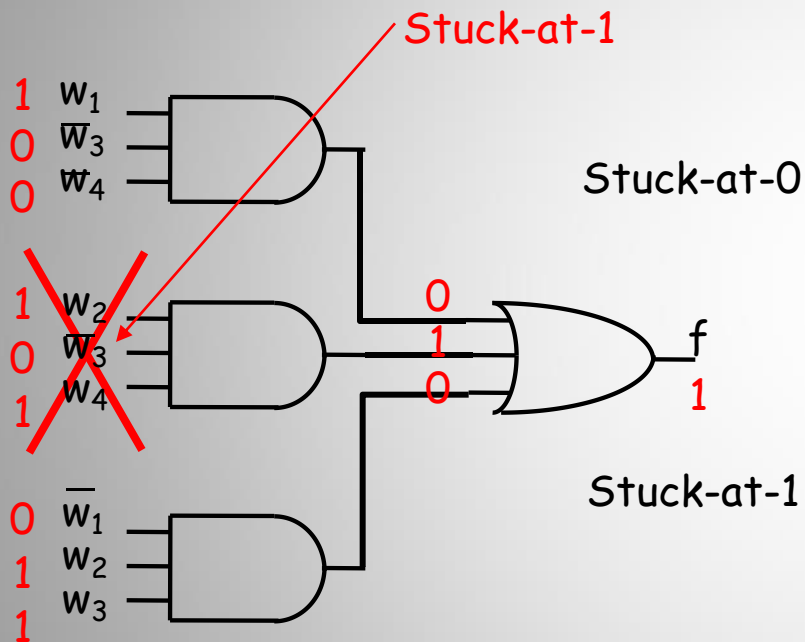
Tree Structured Circuits



	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Any other stuck-at-1 cases covered?

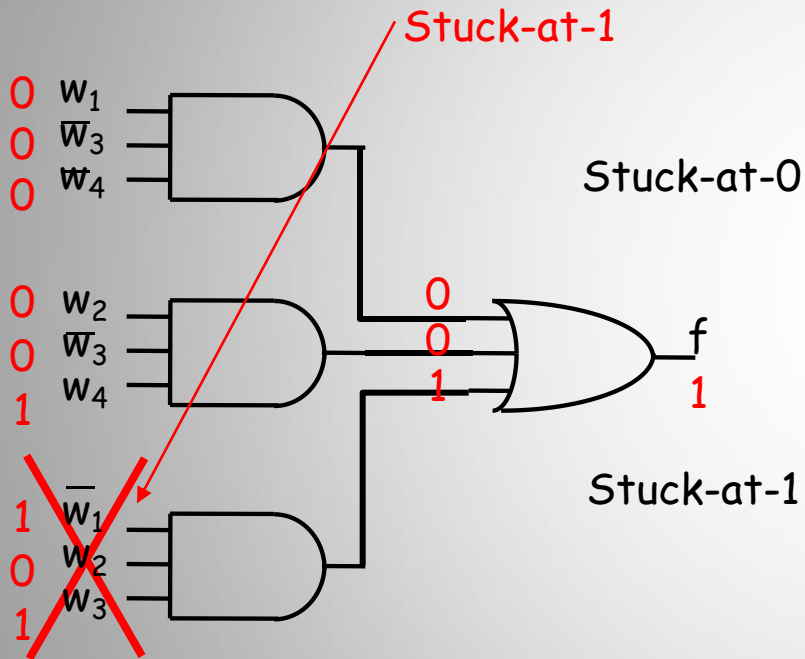
Tree Structured Circuits



	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

Any other stuck-at-1 cases covered?
Was that case already covered?

Tree Structured Circuits



	Product Term									Test			
	w_1	$\bar{w}_3$	$\bar{w}_4$	w_2	$\bar{w}_3$	w_4	$\bar{w}_1$	w_2	w_3	w_1	w_2	w_3	w_4
1	1	1	1	0	1	0	0	0	0	1	0	0	0
2	0	1	0	1	1	1	1	1	0	0	1	0	1
3	0	0	0	1	0	1	1	1	1	0	1	1	1
4	0	1	1	1	1	0	1	1	0	0	1	0	0
5	1	0	1	1	0	0	0	1	1	1	1	1	0
6	1	1	0	0	1	1	0	0	0	1	0	0	1
7	1	0	0	1	0	1	0	1	1	1	1	1	1
8	0	0	0	0	0	1	1	0	1	0	0	1	1

All inputs stuck-at-1's covered now

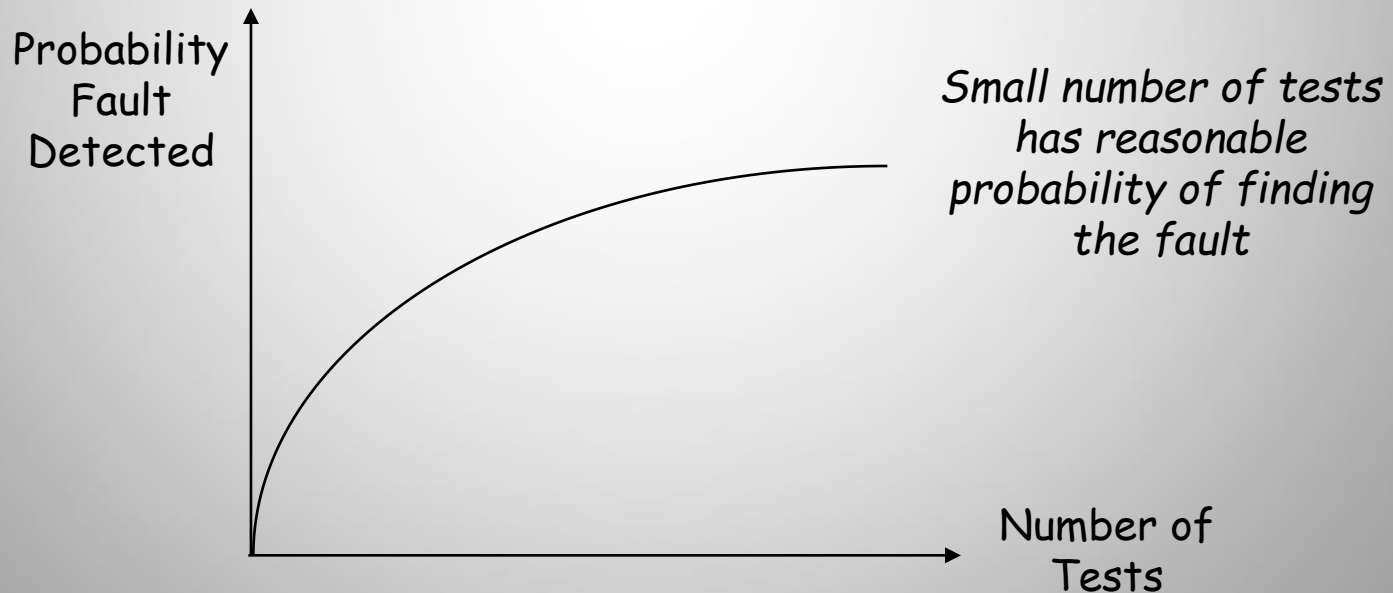
Random Testing

- **So far: deterministic testing(确定性)**

- 电路中的同一个缺陷为许多输入图形所覆盖，检测出这样一个缺陷只需要这些矢量中的一个，而其他矢量是冗余的。

- **Alternative: random testing**

- Generate random input patterns to distinguish between the correct function and the faulty function
- 放宽必须检测出所有缺陷这一要求可以大大减少矢量的数目。为此一般的测试过程只要求95-99%的故障覆盖率。



Sequential Testing

- Due to embedded state inside flip-flops, it is difficult to employ the same methods as with combinational logic
 - 为了测试一个状态机中一定的故障仅仅应用正确的输入激励是不够的。
 -
 - 因为首先必须使这个被测部件处于所希望的状态。这需要应用一系列的输入。同时把电路响应传送到其中的输出上。
 - 测试一个FSM中的单个缺陷需要一系列的测试向量。
- Alternative approach: design for test
 - Scan Path technique:
 - » FF inputs pass through multiplexer stages to allow them to be used in normal mode as well as a special test shift register mode
 - » 在测试过程中把反馈回路断开，从而把时序电路变成组合电路。
 - 自测试 (self-test)

设计可测试性的重要特性

- 设计可测试性的重要特性
 - 高的可控性。
 - » 只利用输入引线就可以使一个电路节点进入某一指定状态
 - » 如果只用一个输入向量就可以把一个节点带到任何状态，那么容易控制
 - » 一个具有低可控性的节点或电路需要一个很长的向量序列才能到预期的状态。
 - 高可观察性
 - » 在输出引线上观察一个节点的值
 - » 对于一个具有高可观察性的节点，可以在输出引线上直接监测到它的值
 - » 一个低可观察性的节点则需要多个周期才能使的状态出现在输出口上。
 - » 当电路的复杂性和引线数目一定时，一个可测电路应当具有较高的可观察性。
- 专门测试（ad hoc test）
 - 同应用类型相关
 - 集合了一些可用来提高设计的可观察性和可控性的技术
- 额外的I/O引线
 - 为了减少可能需要的额外压焊块的数目，可以采用在同一个压焊块上分路选择测试信号和功能信号的方法。
 - I/O总线在正常工作期间作为数据总线，而在测试期间则用来提供测试图形和收集响应。

Design for Testability

- **Basic idea**

- Convert a sequential circuit into a combinational circuit
 - » Control every flip-flop content
 - » Observe every flip-flop output

- **How?**

- Connect flip-flops into one or more shift registers

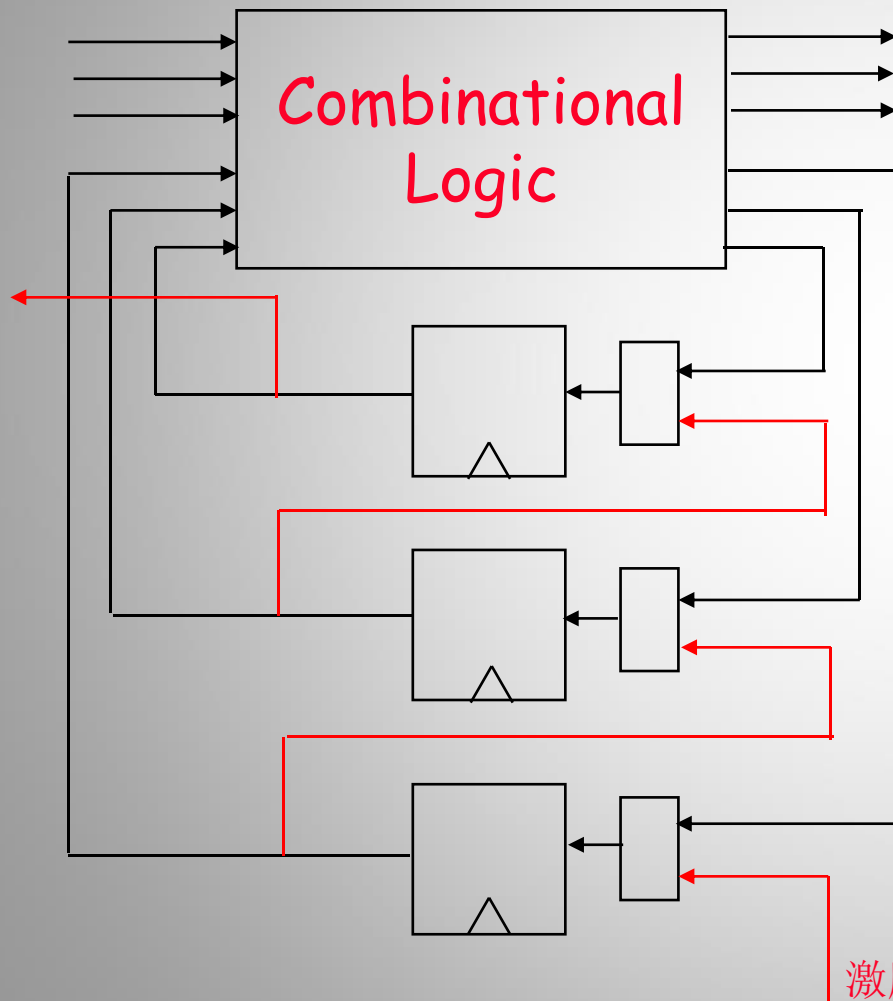
- **SCAN Design for Testability (Scan DFT)**

- **Other benefits**

- Diagnosis & debug
- Low cost test equipment

Scan Path Technique 扫描测试

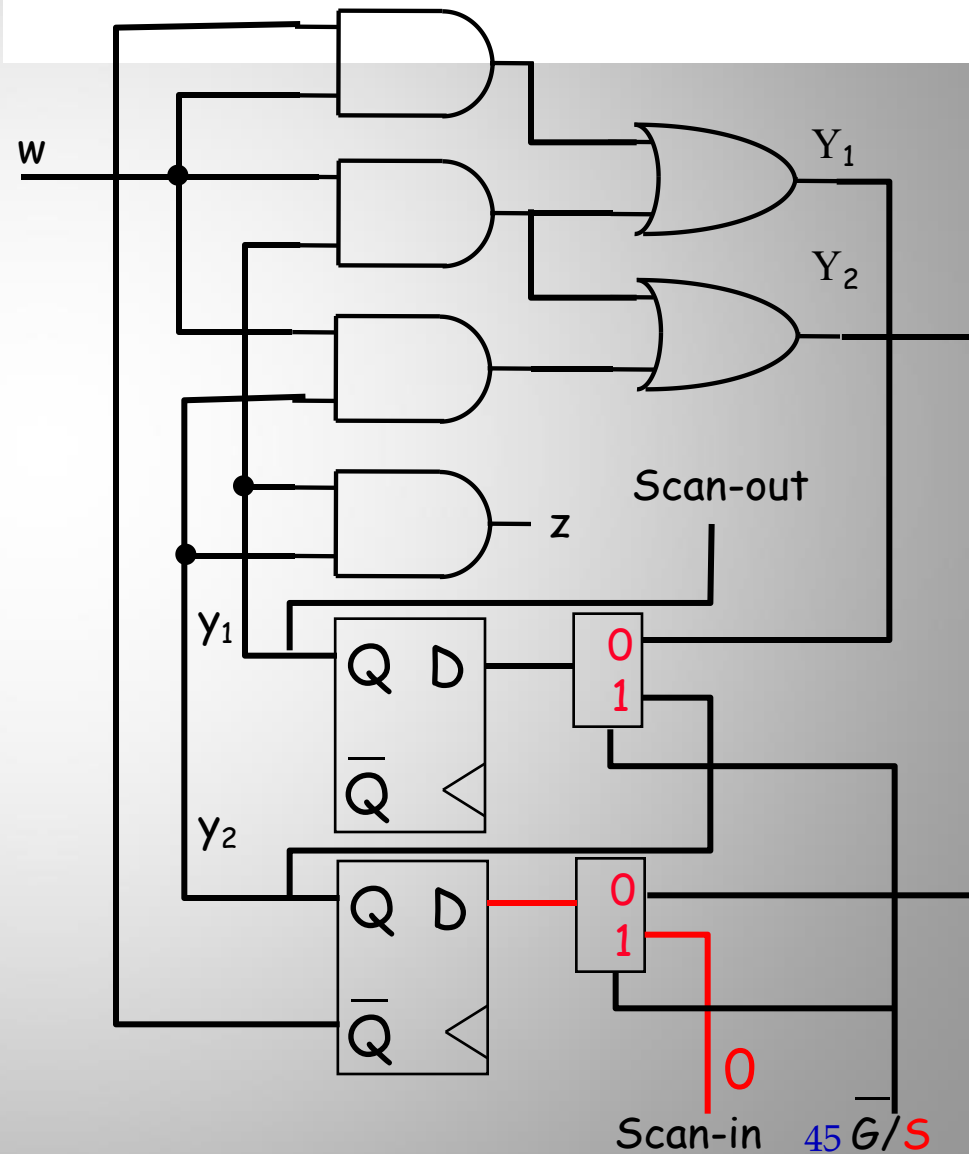
串联扫描方式



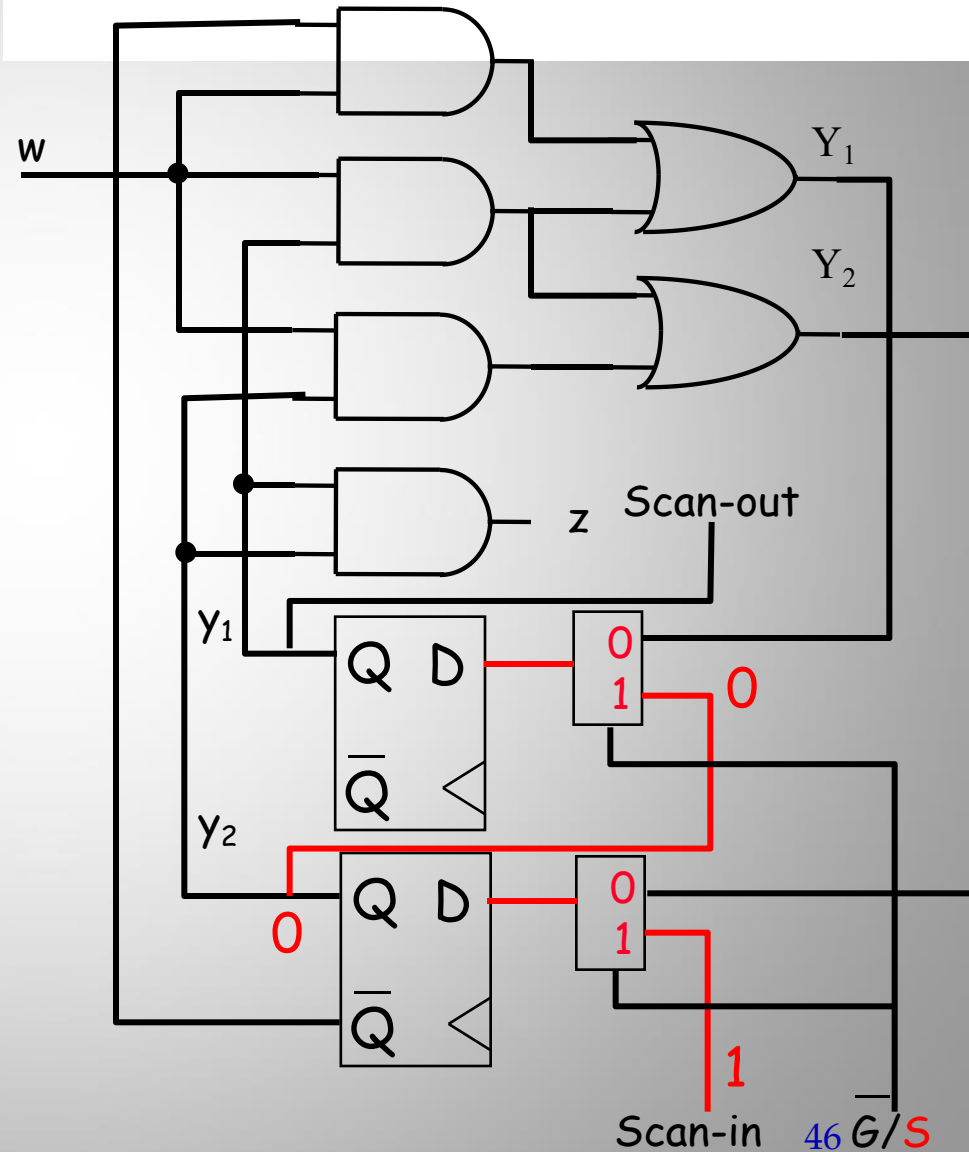
- **Configure** FFs into **shift register** mode (**red path**)
- **Scan** in test pattern of 0s and 1s
- **Non-state inputs** can also be on the scan path (think **synchronous Mealy Machine**)
 - 激励向量通过引线ScanIn输入逻辑模块
- **Run system** for one clock cycle in “normal” mode (**black path**)—**next state captured in scan path**
 - 结果锁存到寄存器中
- **Return to shift register mode and shift out the captured state and outputs**
 - 寄存器中的结果通过引线ScanOut送出电路并与期望的数据进行比较。

Scan Path Example

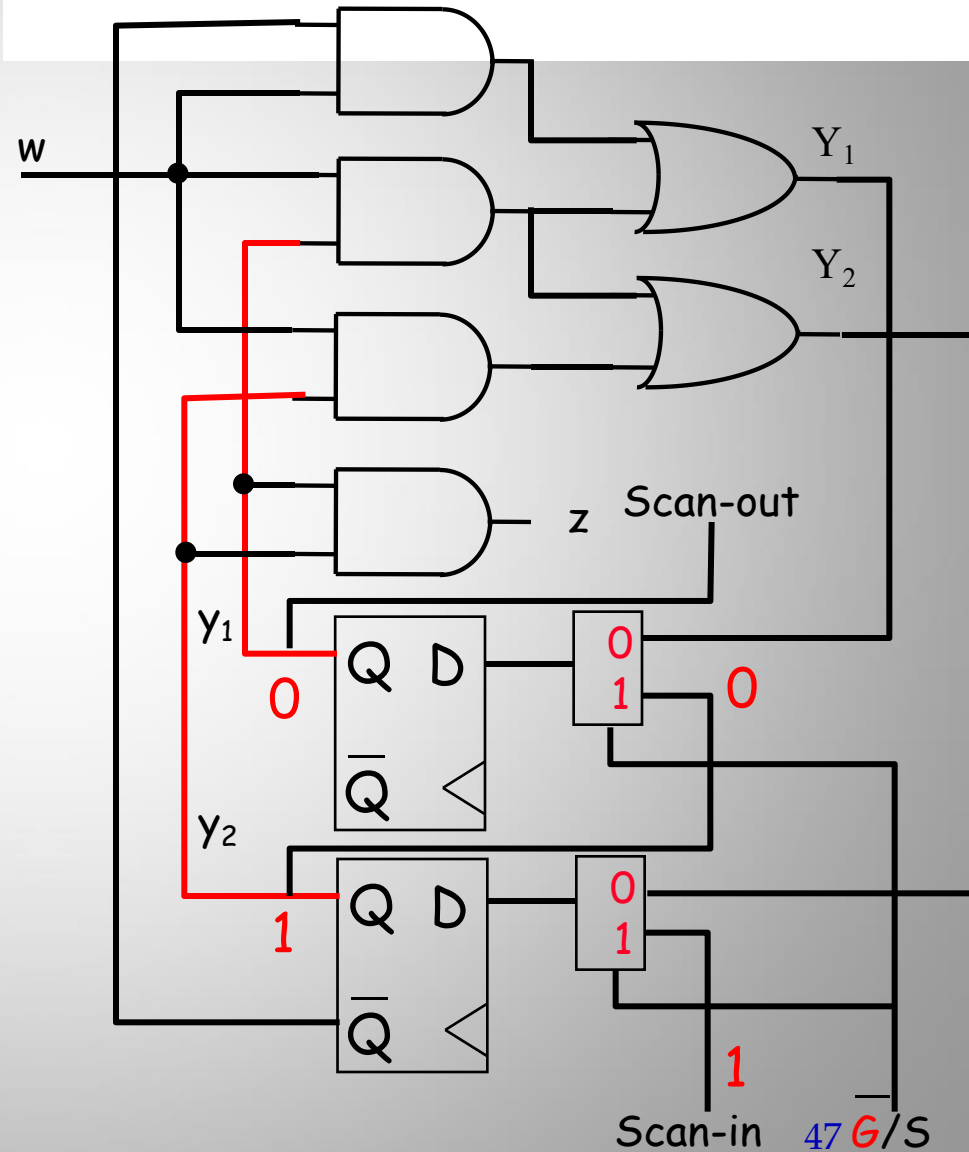
- w, y_1, y_2 test vector 001
 - Scan 01 into y_1, y_2 FFs



- w,y1,y2 test vector 001
 - Scan 01 into y1, y2 FFs

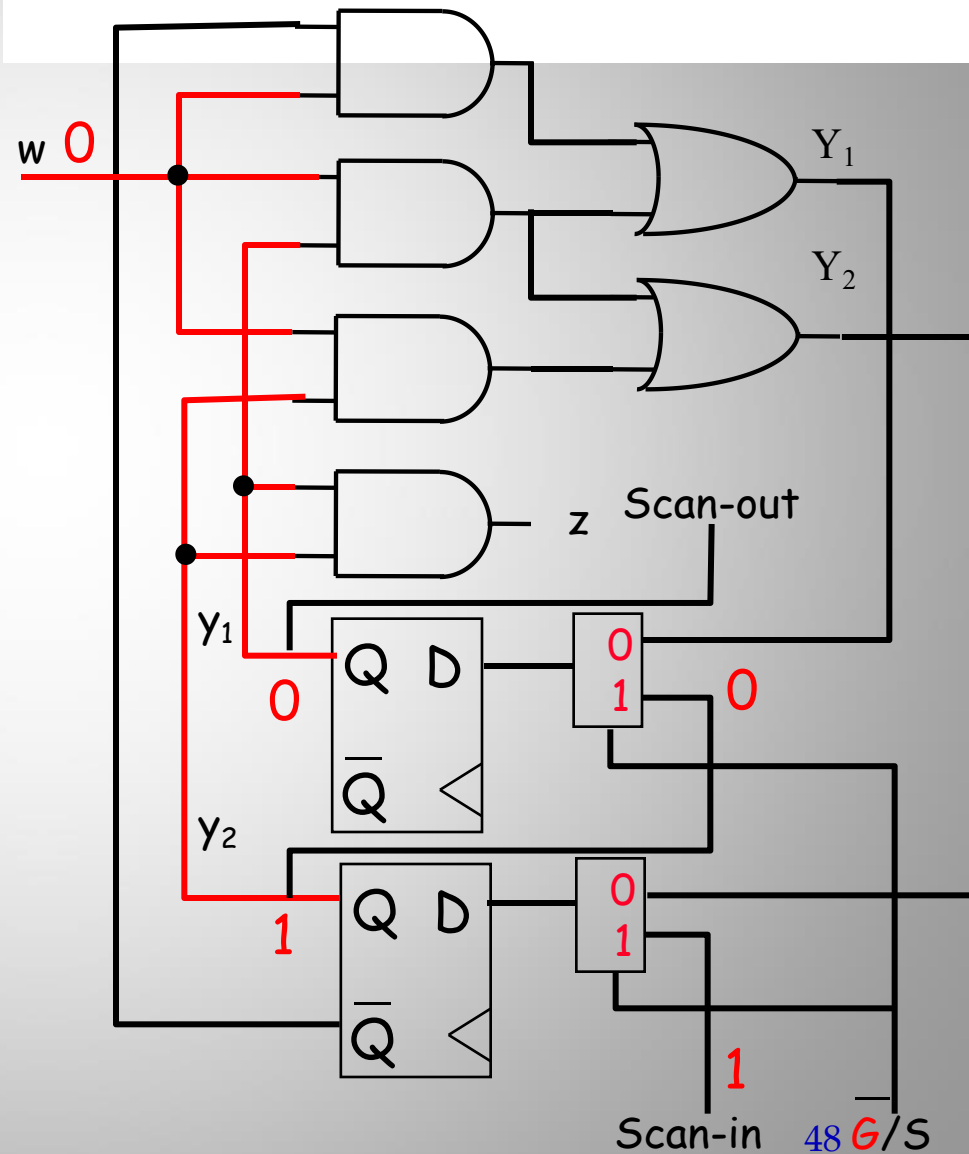


- w,y1,y2 test vector 001
 - Scan 01 into y1, y2 FFs



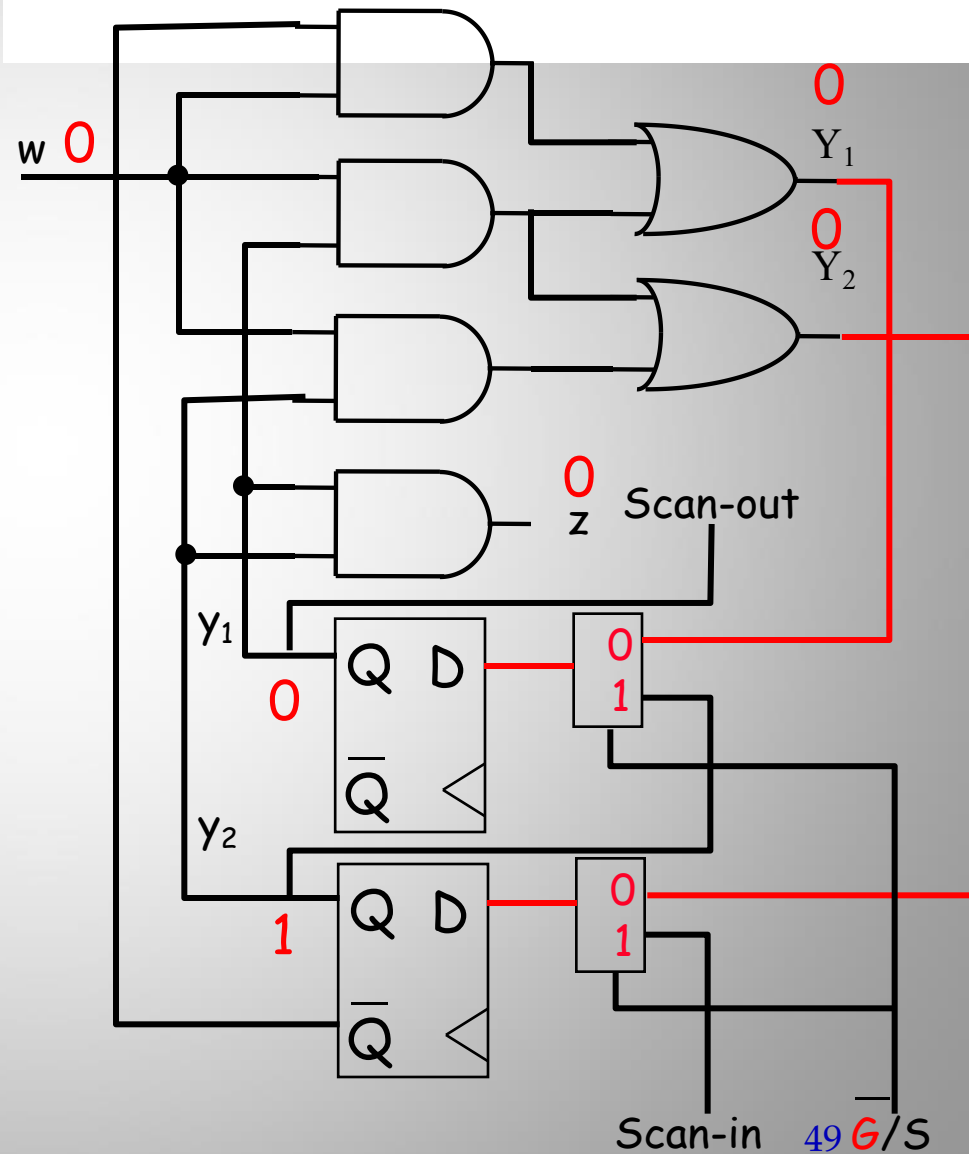
Scan Path Example

- w,y1,y2 test vector 001
 - Scan 01 into y1, y2 FFs
 - Normal w=0



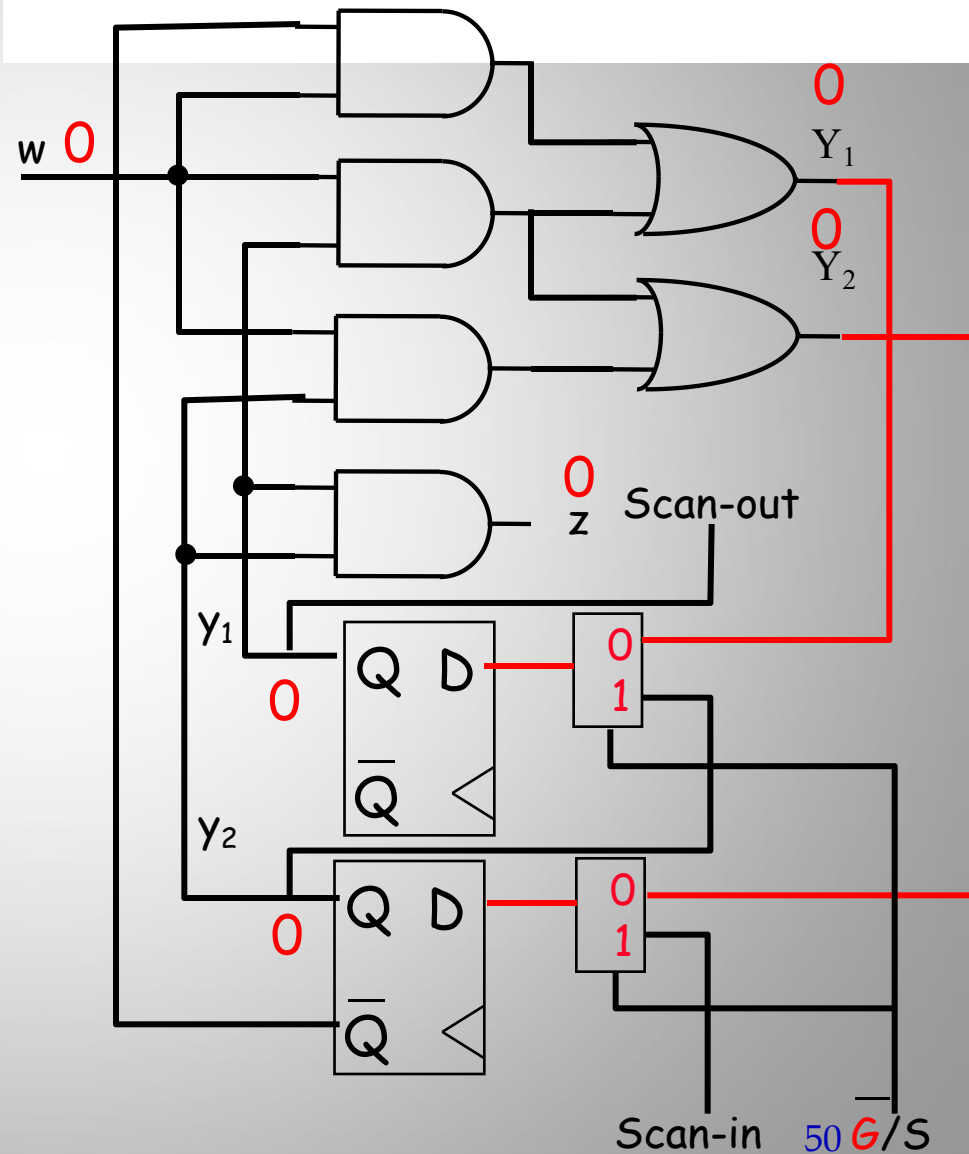
Scan Path Example

- w,y1,y2 test vector 001
 - Scan 01 into y1, y2 FFs
 - Normal w=0
 - Output z=0, Y1=0, Y2=0

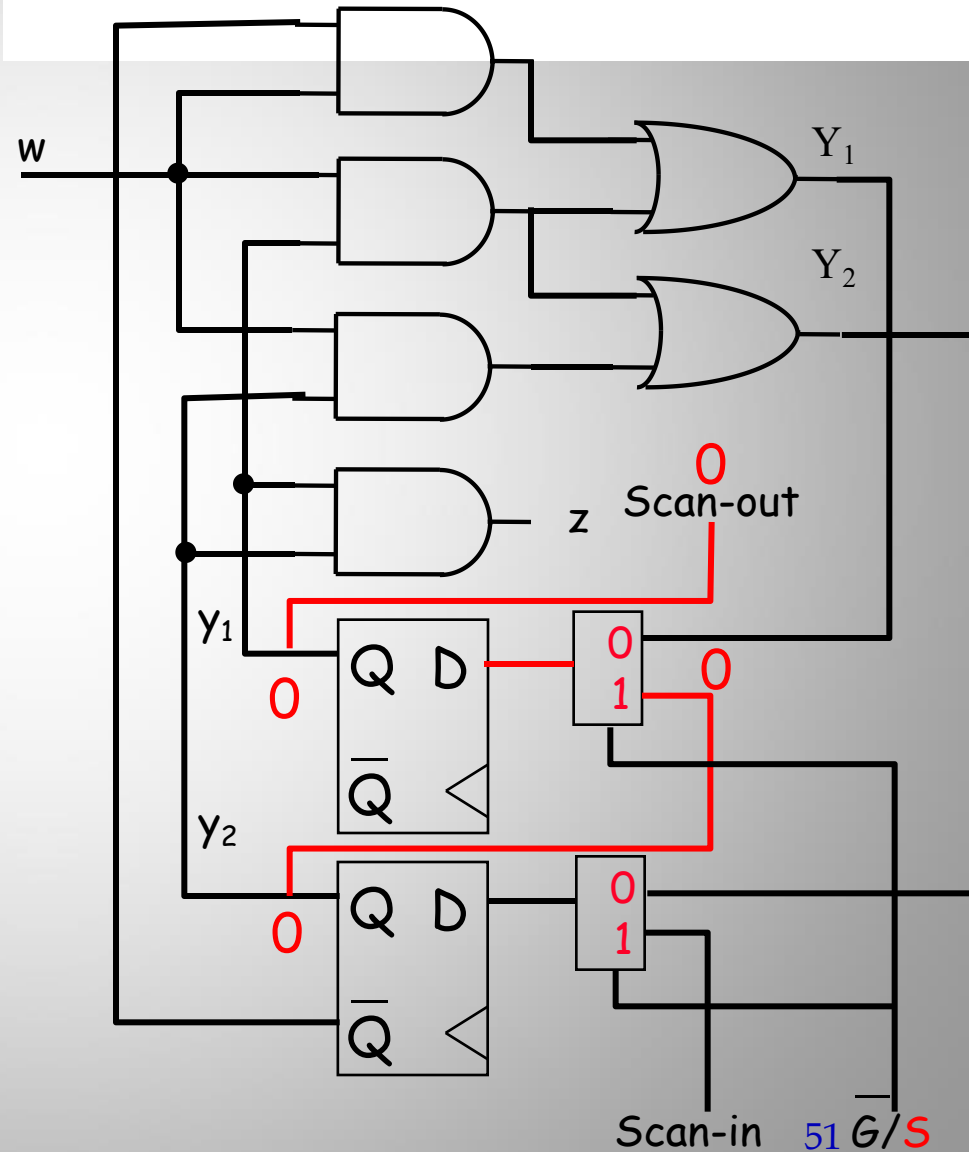


Scan Path Example

- w, y_1, y_2 test vector 001
 - Scan 01 into y_1, y_2 FFs
 - Normal $w=0$
 - Output $z=0, Y_1=0, Y_2=0$
 - Observe z directly

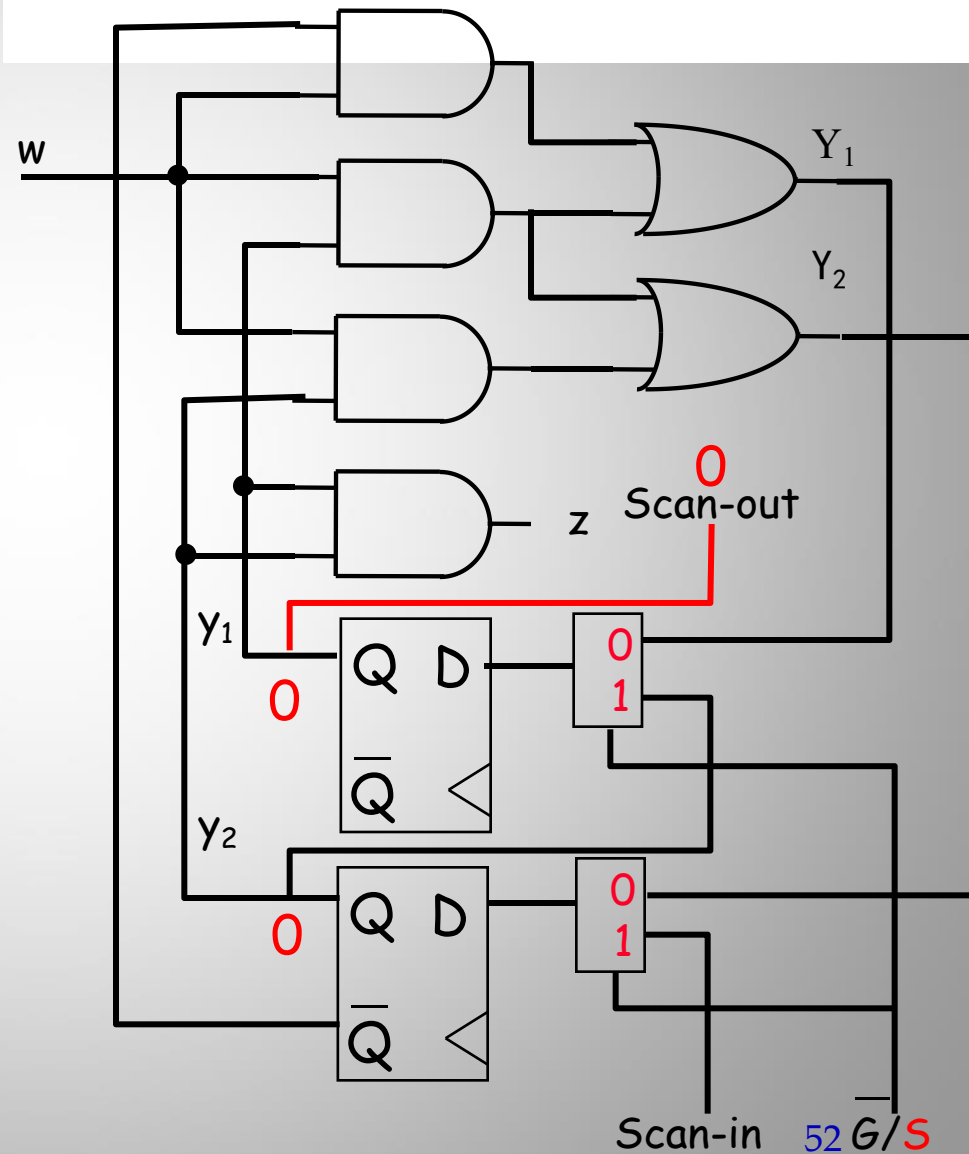


- w, y1, y2 test vector 001
 - Scan 01 into y1, y2 FFs
 - Normal w=0
 - Output z=0, Y1=0, Y2=0
 - Observe z directly
 - Scan out Y1, Y2



Scan Path Example

- w, y_1, y_2 test vector 001
 - Scan 01 into y_1, y_2 FFs
 - Normal $w=0$
 - Output $z=0, Y_1=0, Y_2=0$
 - Observe z directly
 - Scan out Y_1, Y_2



Scan Operations

- Scan-in: test pattern shifted in
 - **Apply Test Mode = 1**
 - **Number of clock cycles applied in this mode = number of flip-flops in scan chain (shift register)**
- • Apply primary inputs
- • Capture: combinational logic response to the test pattern captured
- in flip-flops
 - **Apply Test Mode = 0, and 1 or more clock cycles**
- • Observe primary outputs
- • Scan-out: captured response shifted out
 - **Apply Test Mode = 1**
 - **Number of clock cycles applied in this mode = number of flip-flops in scan chain (shift register)**

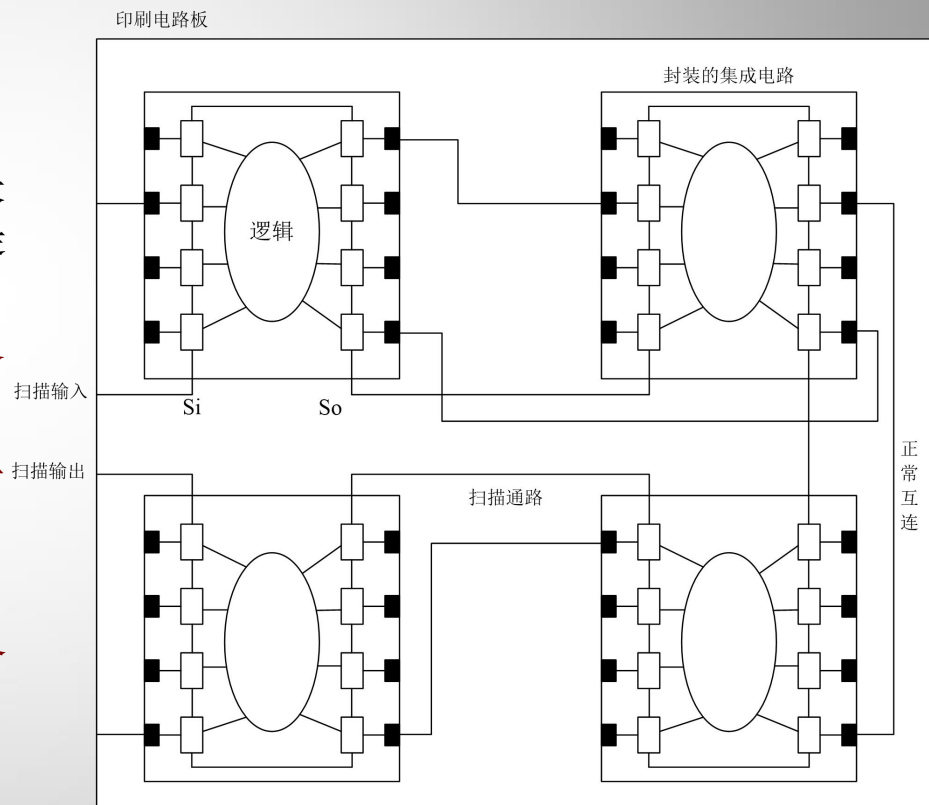
部分扫描

- 并不是设计中所有的寄存器都需要扫描
 - 流水线寄存器只是为了提高性能，并不增加电路的新状态
 - 只需使输入和输出寄存器可扫描就可以了

边界扫描设计 (boundary scan)

• 边界扫描

- 把一个板上部件的输入-输出引线连接成一条串联的**扫描链**
- 在正常工作时，边界扫描压焊块 **pads** 作为正常的**输入-输出器件**
- 在测试模式，向量可以从这些压焊块处**扫入扫出**，从而在部件的边界上提供可控性和可观察性。
 - » 可以利用各种控制模块来测试各个部件以及板上的互连线。
 - » 这方法的开销是要求稍微复杂一些的输入-输出压焊块以及一个附加的片上测试控制器。
 - » 目前大多数产品部件都提供边界扫描。



板级测试的边界扫描方法

边界扫描链

- 边界扫描链
 - 由边界扫描单元串行组成
- 边界扫描单元
 - 能够完成对电路节点的控制和观察功能
- 边界扫描单元的结构

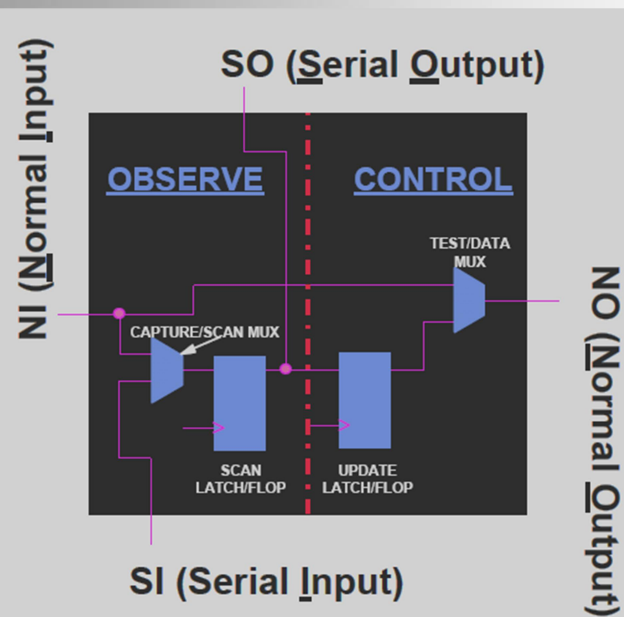


图 8.9 带输出锁存的边界扫描单元

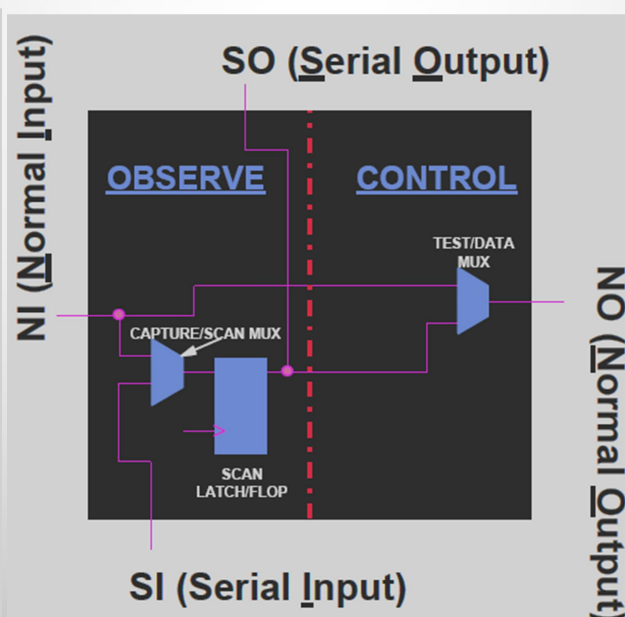


图 8.10 不带输出锁存的边界扫描单元

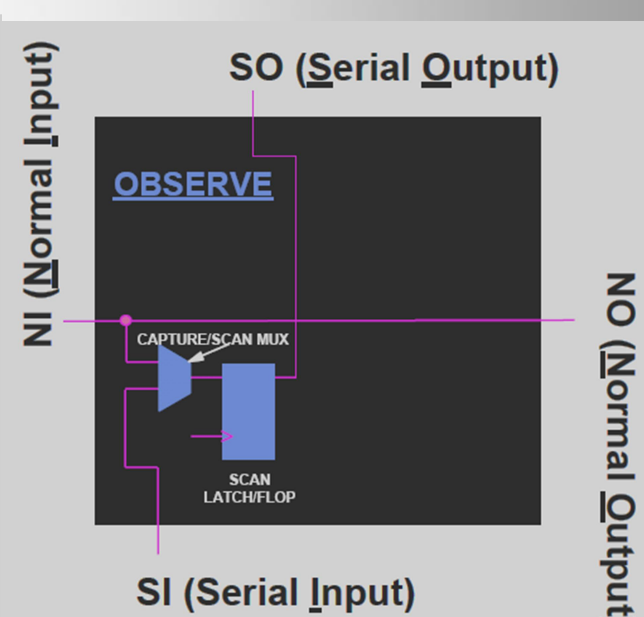


图 8.11 只有观察级的边界扫描单元

- 为了实现PCB测试，芯片的边界扫描链要求必须在所有的管脚添加边界扫描单元。
 - 对于输入管脚、两态输出管脚、三态输出管脚和双向管脚，边界扫描单元添加方式有所不同
 - » 当不实现INTEST指令时，输入管脚的边界扫描单元只用一个观察级的扫描单元即可，
 - » 当需要实现INTEST指令时，还必须添加一个带控制级的扫描单元。

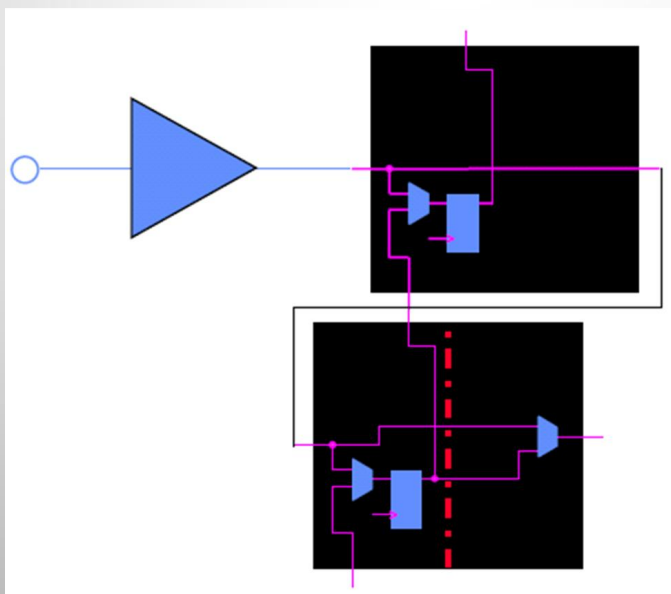


图 8.12 输入管脚的边界扫描单元

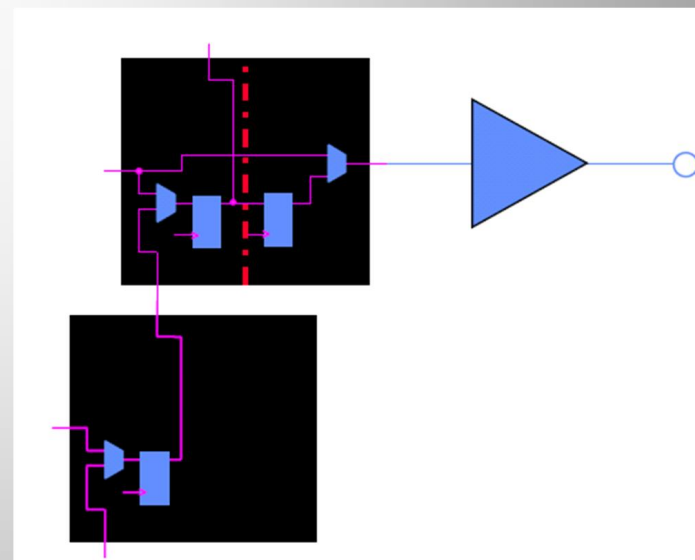


图 8.13 两态输出管脚的边界扫描单元

- 两态的输出管脚必须添加带输出锁存的边界扫描单元

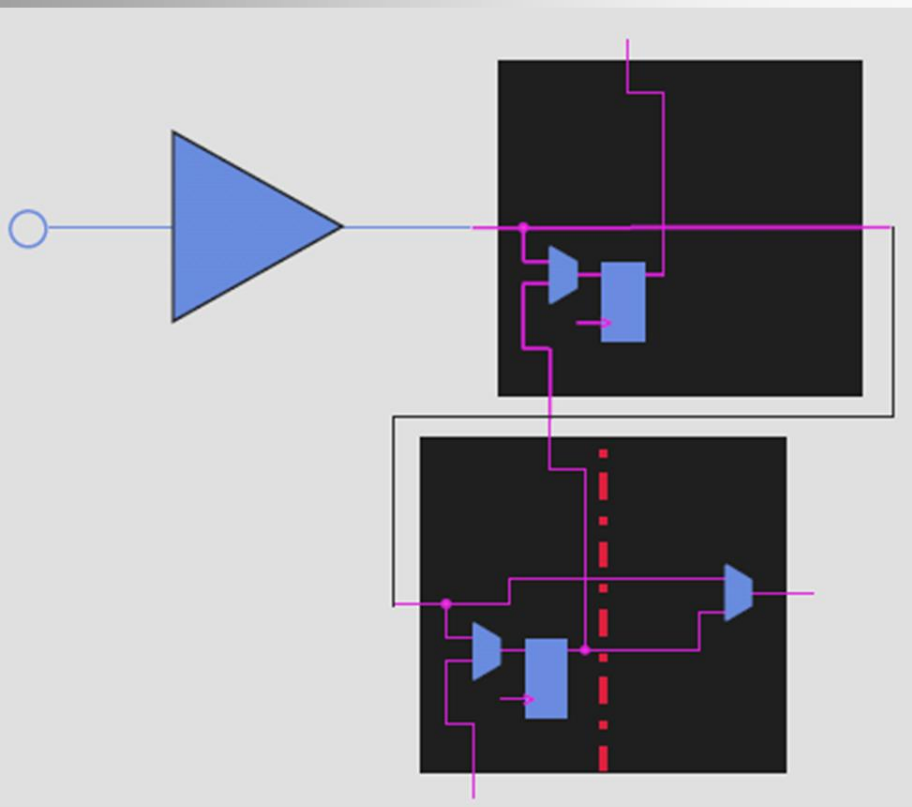


图 8.12 输入管脚的边界扫描单元

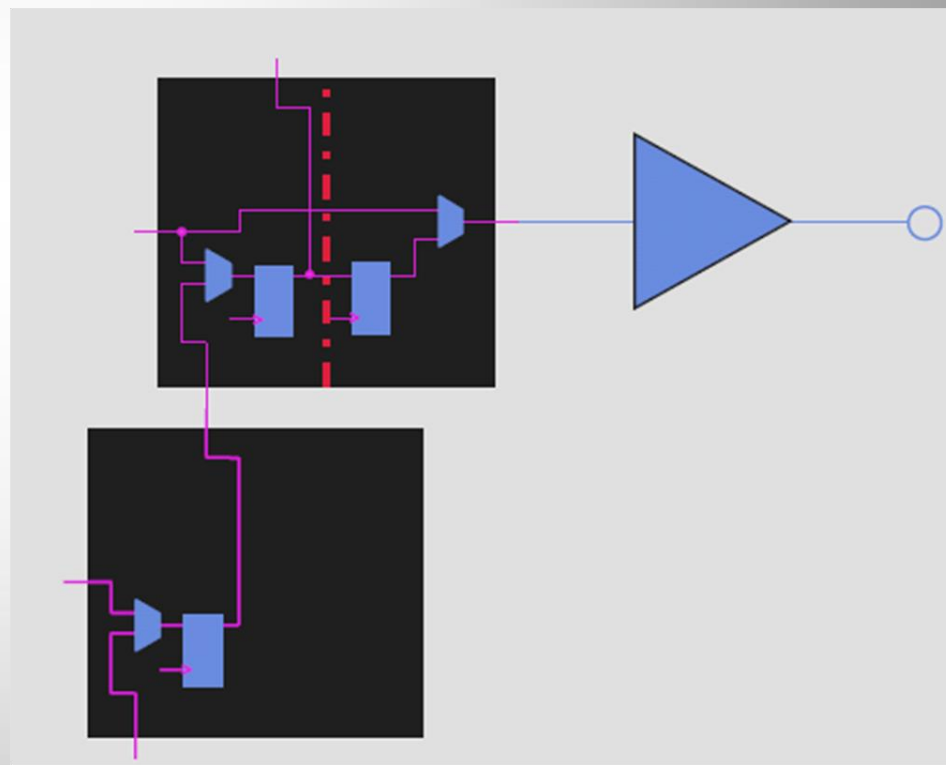


图 8.13 两态输出管脚的边界扫描单元

- 三态输出管脚除了添加带输出锁存的边界扫描单元外，还必须在三态控制信号上也添加一个带输出锁存的边界扫描单元
- 双向边界扫描单元由一个三态输出扫描单元和一个只带观察级的输入扫描单元组成

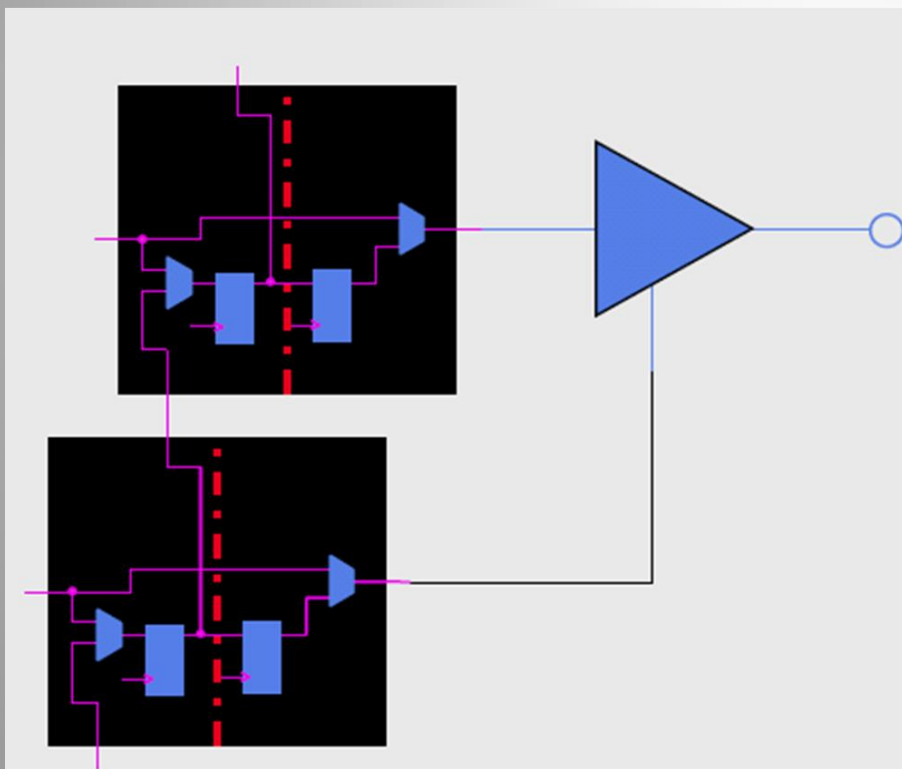


图 8.14 三态输出管脚的边界扫描单元

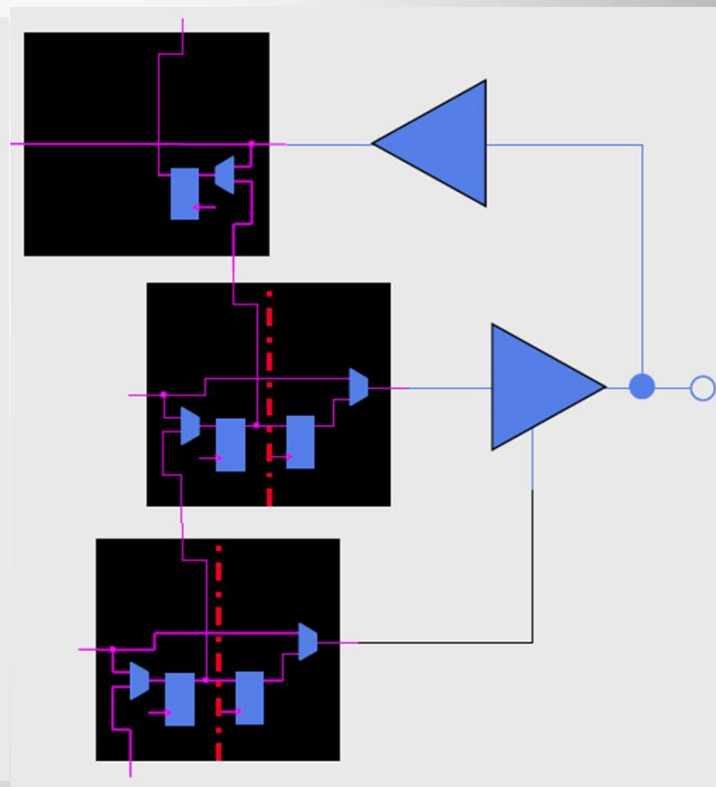
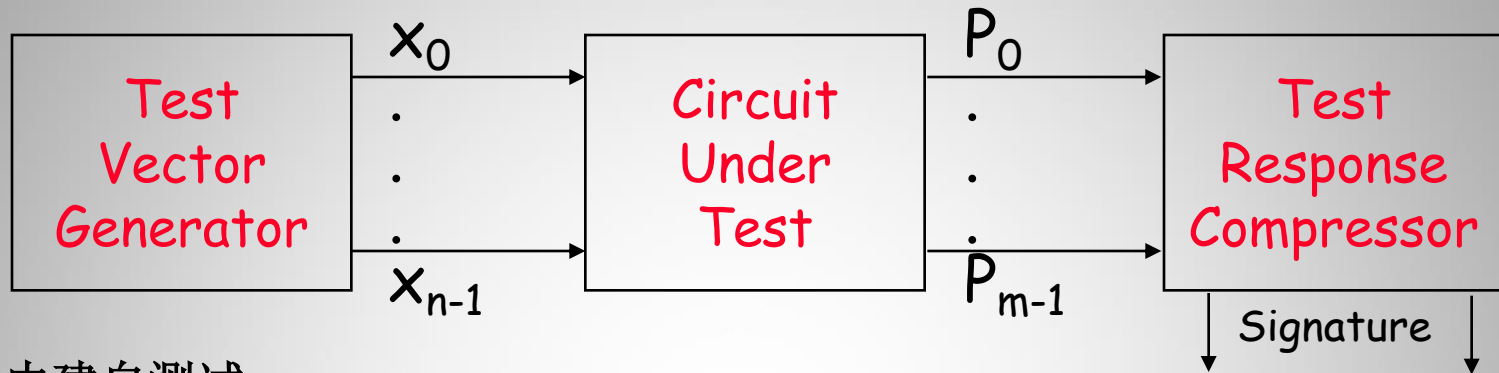


图 8.15 双向管脚的边界扫描单元

边界扫描

- 除了在**管脚**上添加边界扫描单元外，为了实现芯片内部的电路级可测试性，还必须在内部的电路节点添加扫描单元。
- **扫描单元**一般添加在寄存器的位置，用带扫描功能的寄存器替换设计中的所有寄存器，即可实现全扫描设计。
- **全扫描资源**消耗很大，同时扫描链过长也使得串行数据扫描的速度急剧下降，使得测试速度变慢。
- 需要根据电路逻辑结构采用部分扫描设计。

Built-in Self-Test (BIST)内建测试

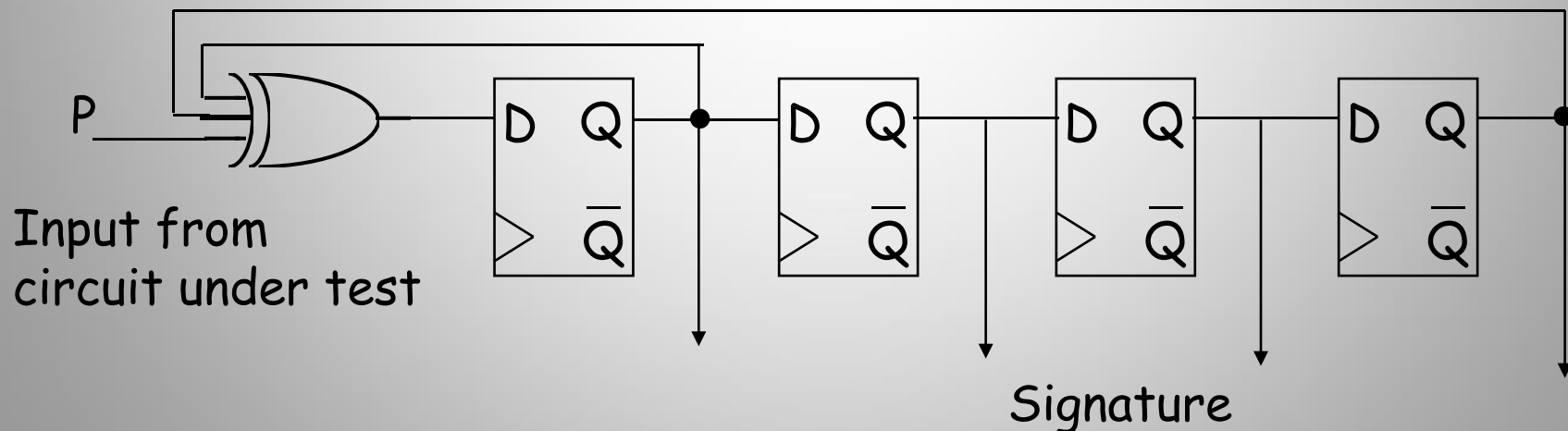
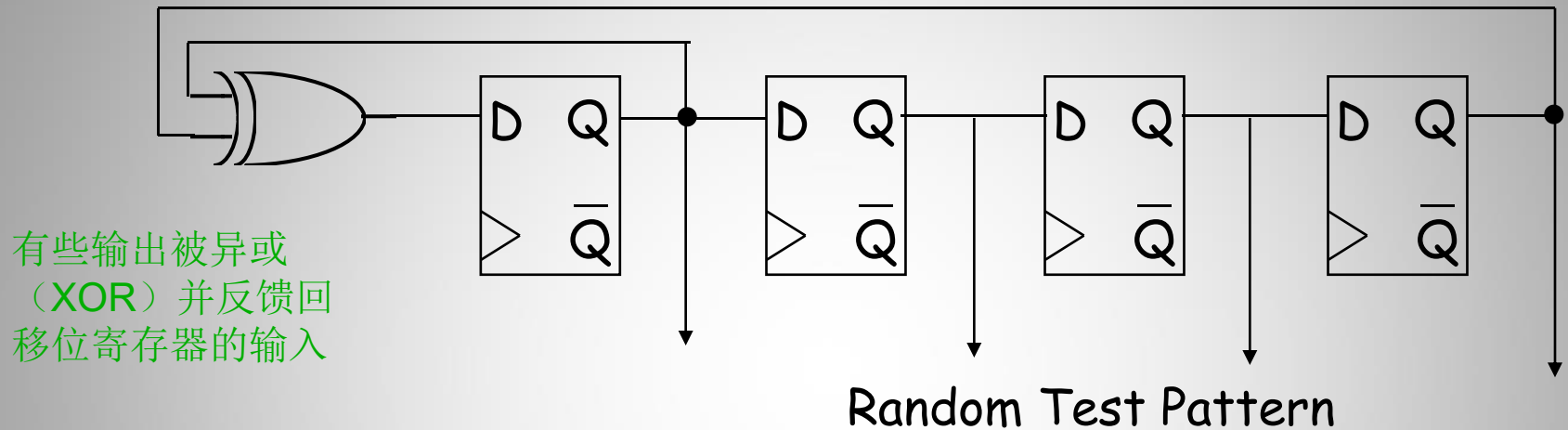


- 内建自测试
 - 让电路自己生成测试图形
 - 电路自己能够决定它所得到的测试结果是否正确
- Test Vector Generator
 - Pseudorandom tests with a feedback shift register
 - » 穷尽测试，所有可以得到的输入信号空间，所有可测的故障都会被检测到。
 - » 对于N值较大的电路，通过整个输入空间的操作所需要的时间是无法接受的。
 - Seed generates a sequence of test patterns
 - Outputs combined using the same technique
 - Generates a unique signature签名 that can be checked to determine if the circuit is correct

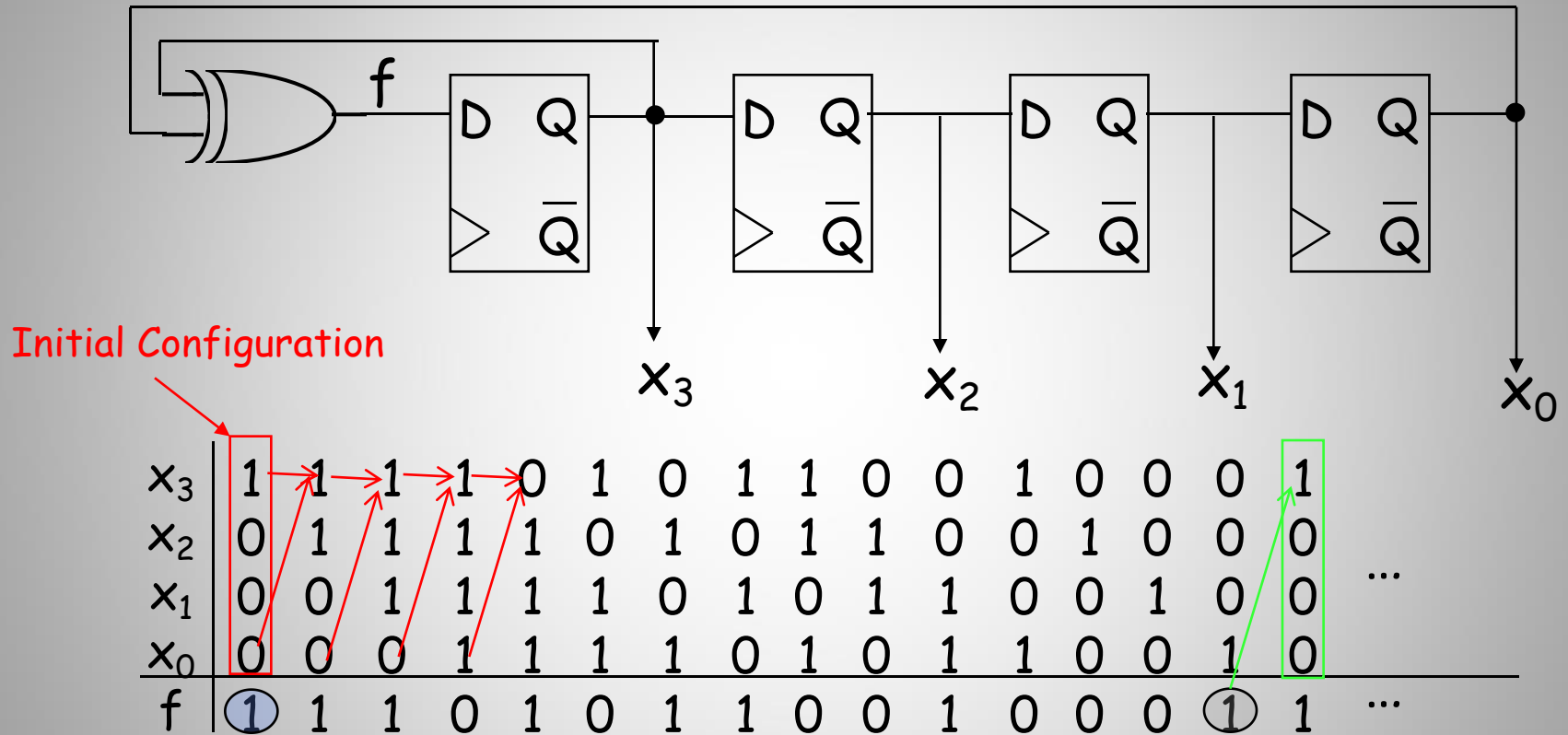
原则是应当能得到合理的故障覆盖率

Linear Feedback Shift Register

由多个一位的寄存器串联构成。把这个寄存器初始化为一定的种子值就会产生不同的伪随机序列。



Linear Feedback Shift Register



- Starting with the pattern 1000种子, generates 15 different patterns in sequence and then repeats
- Pattern 0000 is a no-no

Linear Feedback Shift Register

改进的线性反馈移位寄存器

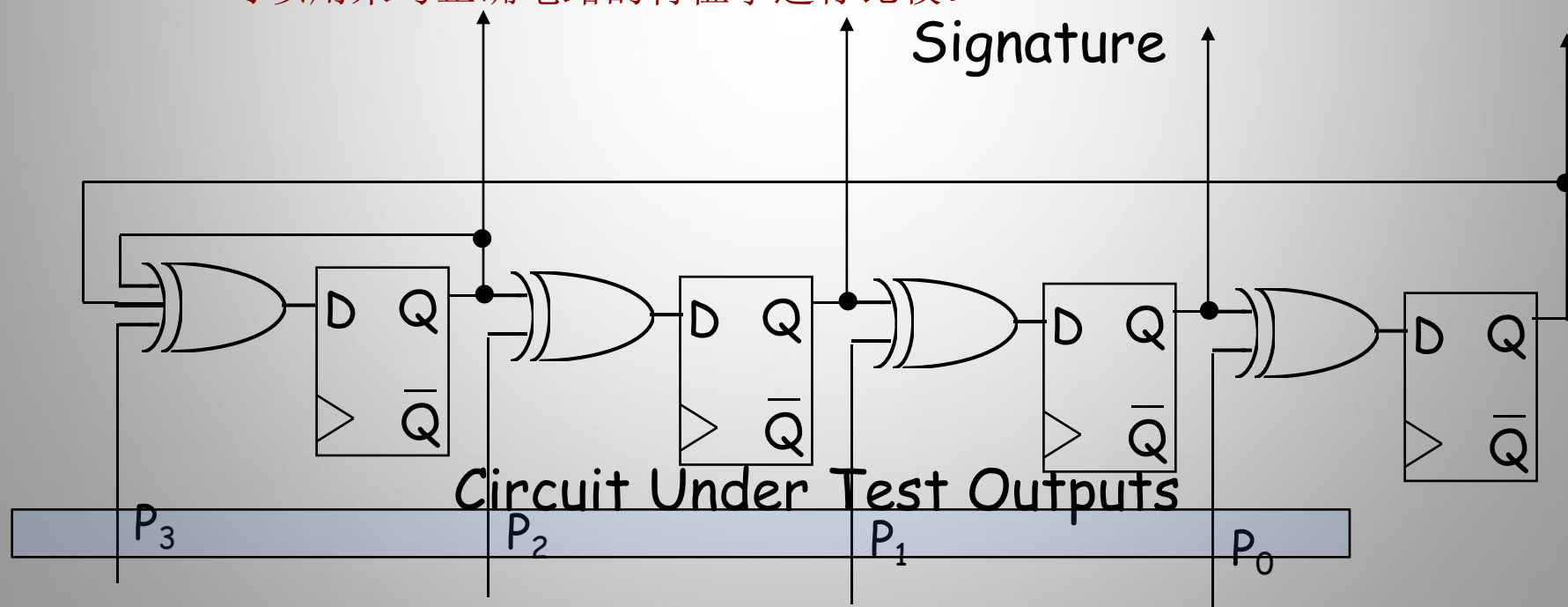
用同一硬件来完成图形生成和信号分析

- 响应分析器

- 将所生成的响应与存放在片上存储器中的预期响应进行比较
- 更经济的技术是在对它们进行比较之前把这些响应进行压缩

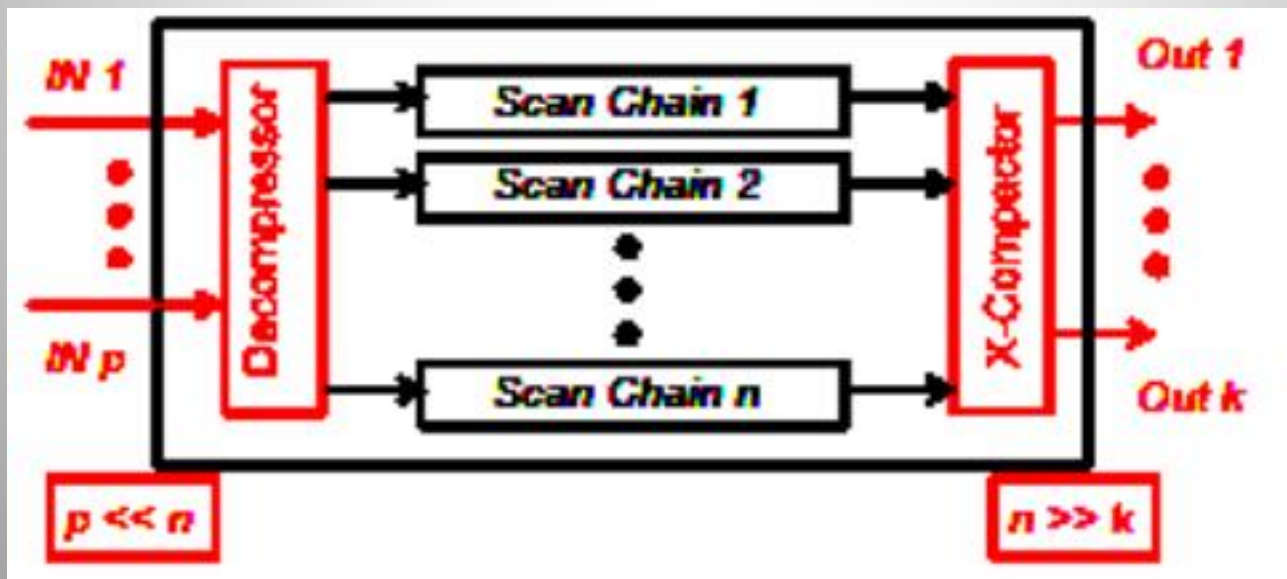
- Multi-input Compressor

- 每一个进来的数据字被一个接一个地与LFSR的内容相比较(XOR)。
- 在测试序列结束时，LFSR中含有了这个数据序列的签名或特征字，可以用来与正确电路的特征字进行比较。



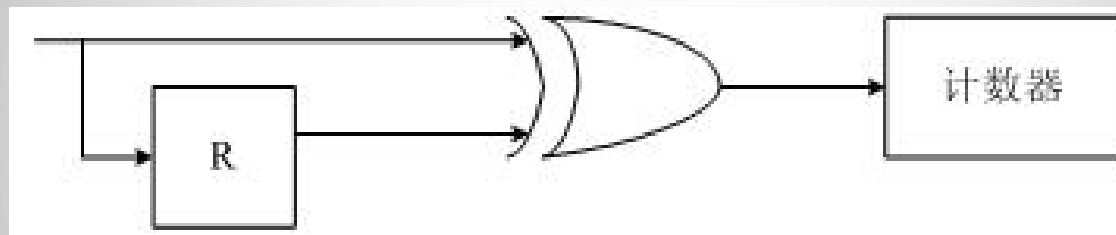
Test Compression

- • BIST benefits retained, BIST issues avoided
- • MAJOR proliferation in the industry
- • Example configurations:
 - $n = 200, p = k = 15$;
 - $n = 500, p = k = 20$;
 - $n = 1,000, p = k = 50$



- 一位信号流的压缩签名分析器

- 响应分析器有一个动态压缩被测电路输出的电路以及一个比较器构成。
- 被压缩的输出也常常称为该电路的签名，而整个方法称为签名分析。

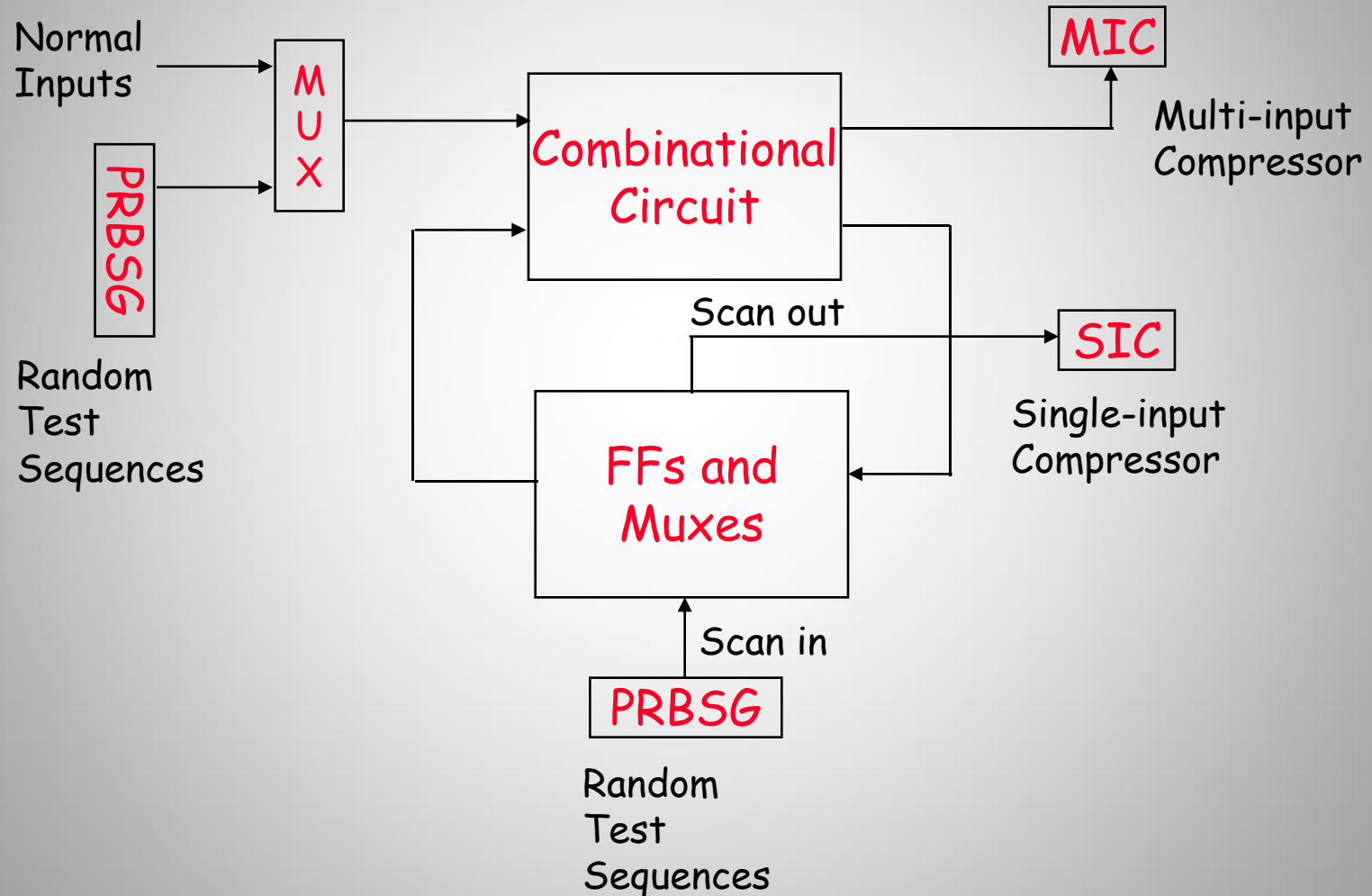


- 这一电路只是计数输入流中0->1和1->0的翻转数目。这一压缩并不能保证接收到的序列是正确的，也就是说许多不同的序列可以有相同数目的翻转。
- 但是翻转数目不对，电路肯定有错。

Test Compression Advantages

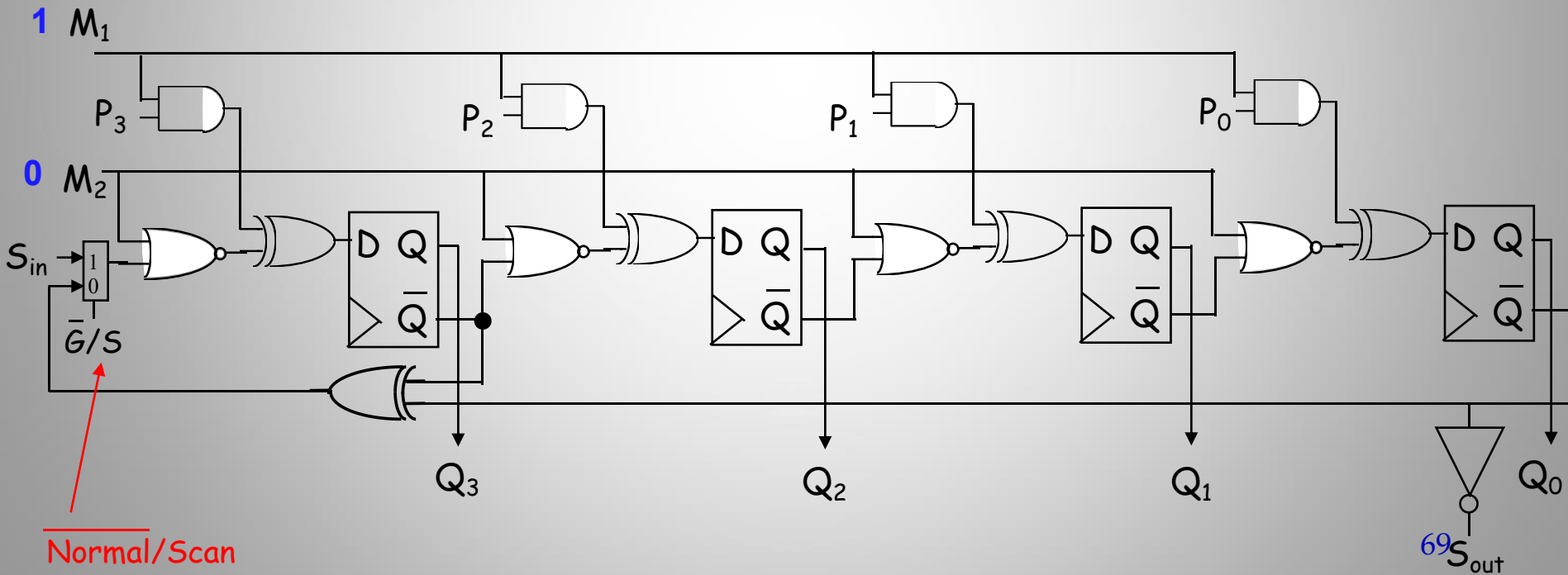
- • Exponential reduction (50 – 80X, sometimes even more)
 - Test data volume, time, pins, tester channels
- • High test quality
- • **X tolerance**
 - New X-Compactor technique – an example of combining VLSI
- with information theory
- • Commercial ATPG tool compatibility
- • Diagnosis support
- • Fault model & test set independence
- • Very little area overhead, no extra delay
- • Easy automation

Complete Self-Test System

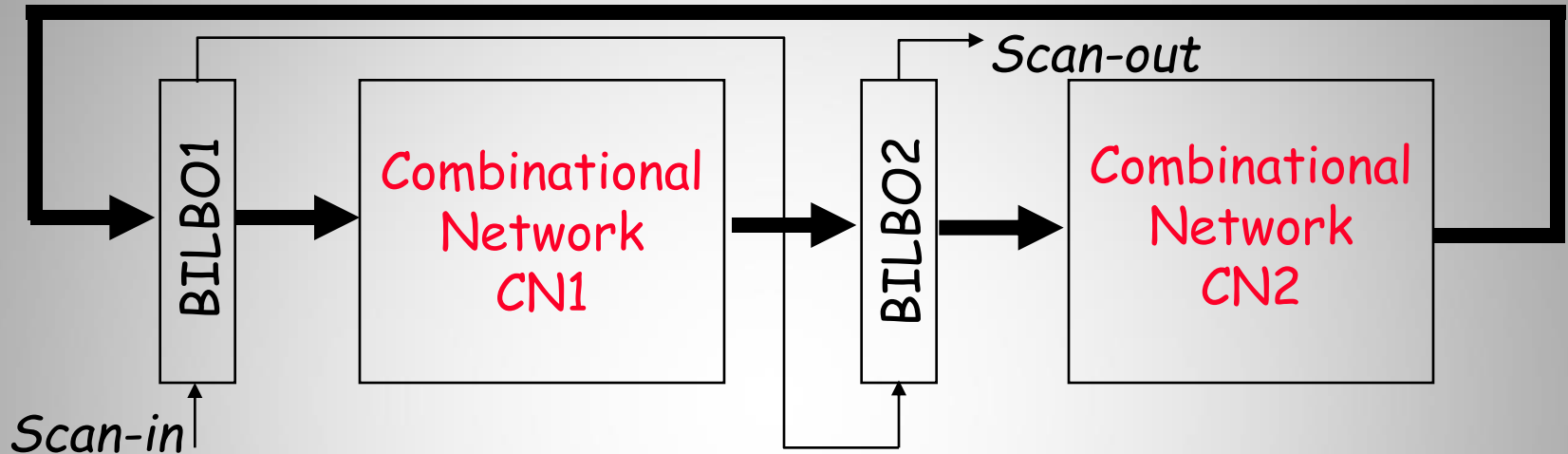


Built-in Logic Block Observer (Bilbo)

- Test generation and compression in a single circuit!
 - $M_1, M_2 = 11$: Regular mode
 - $M_1, M_2 = 00$: Shift register mode 扫描移位寄存器
 - $M_1, M_2 = 10$: Signature generation mode 签名分析/模式产生
 - $M_1, M_2 = 01$: Reset mode

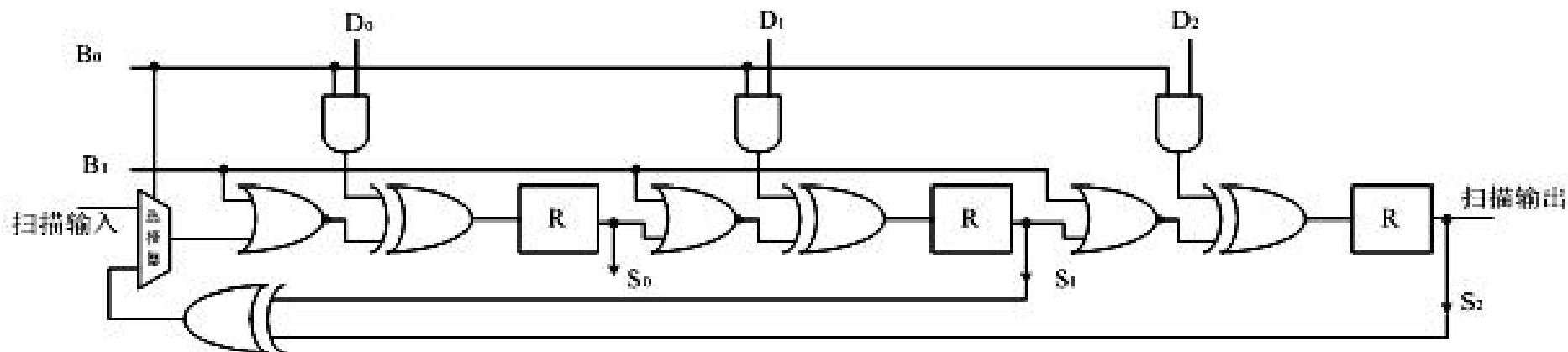


Bilbo Architecture



- Scan initial pattern in Bilbo1, reset FFs in Bilbo2
- Use Bilbo1 as PRBS探测信号 generator for given number of clock cycles and use Bilbo2 to produce signature
- Scan out Bilbo2 and compare signature; Scan in initial test pattern for CN2; Reset the FFs in Bilbo1
- Use Bilbo2 as PRBS generator for a given number of clock cycles and use Bilbo1 to produce signature
- Scan out Bilbo1 and compare signature;

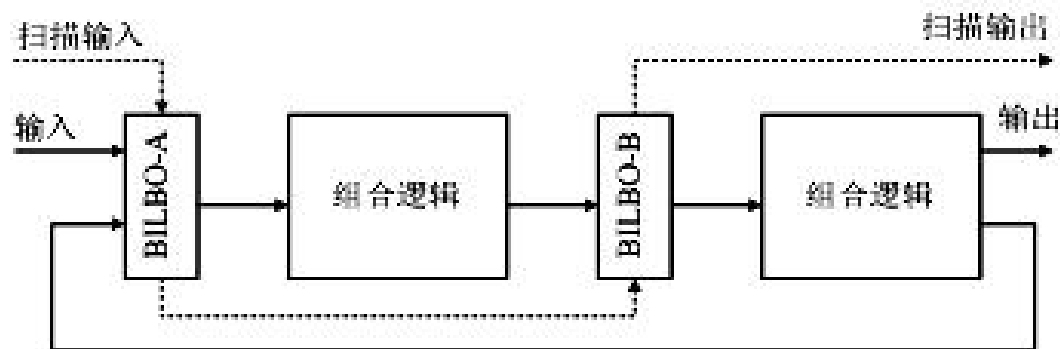
内建逻辑块监测 (BILBO)



(a) 3位BILBO寄存器

$B_0 B_1$	操作模式
1 1	正常
0 0	扫描
1 0	模式产生或签名分析
0 1	复位

(b) BILBO模式

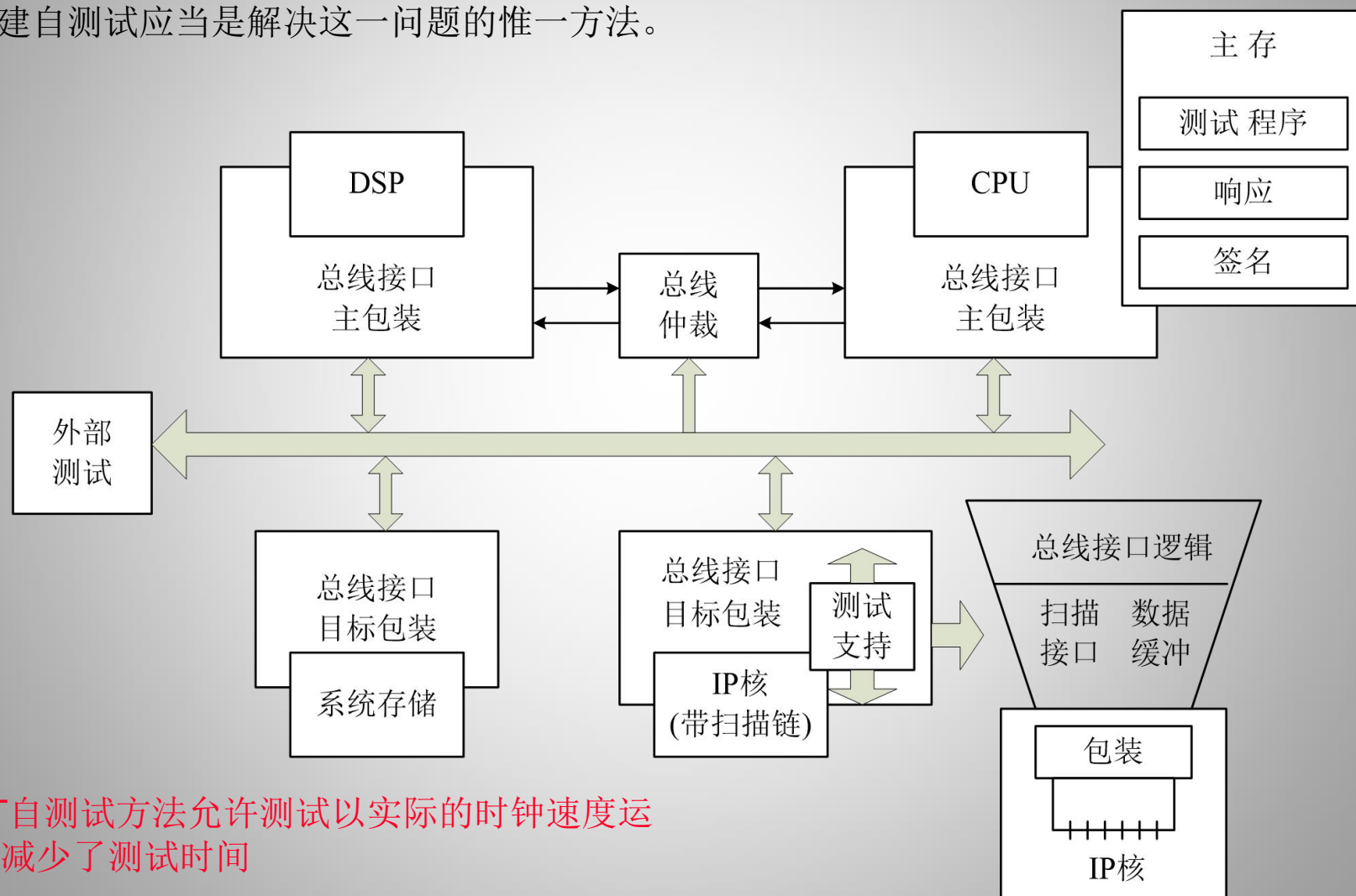


(c) BILBO应用

系统芯片测试方法

片上系统时代的到来并未使测试工作变得更简单一些。

内建自测试应当是解决这一问题的惟一方法。



BIST自测试方法允许测试以实际的时钟速度运行，减少了测试时间

Summary

- **Fault models**故障模型
 - Approach for determining how to develop a test pattern sequence
 - Weakness is the single fault assumption
- **Scan Path**扫描路径
 - Technique for applying test inputs deep within the system, usually for asserting state
 - Technique for getting internal state to edges of circuit for observation
- **Built-in Test**内建测试
 - Founded on the approach of random testing
 - Generate pseudo random sequences; compute signature; determine if signature generated is same as signature of a correctly working circuitry

课后作业

- 1) 查阅:
 - 国际电路公司的
 - » 8051, PIC, AVR单片机, ARM, MIPS, DSP等微控制器芯片的型号、类型、速度.....
 - 图书馆资源: 电子器件天地, 软件
- 2) 作业:
 - 见学在浙大题目
- 3) 阅读:
 - 复习《数字系统设计与Verilog HDL》
 - 书CH.11

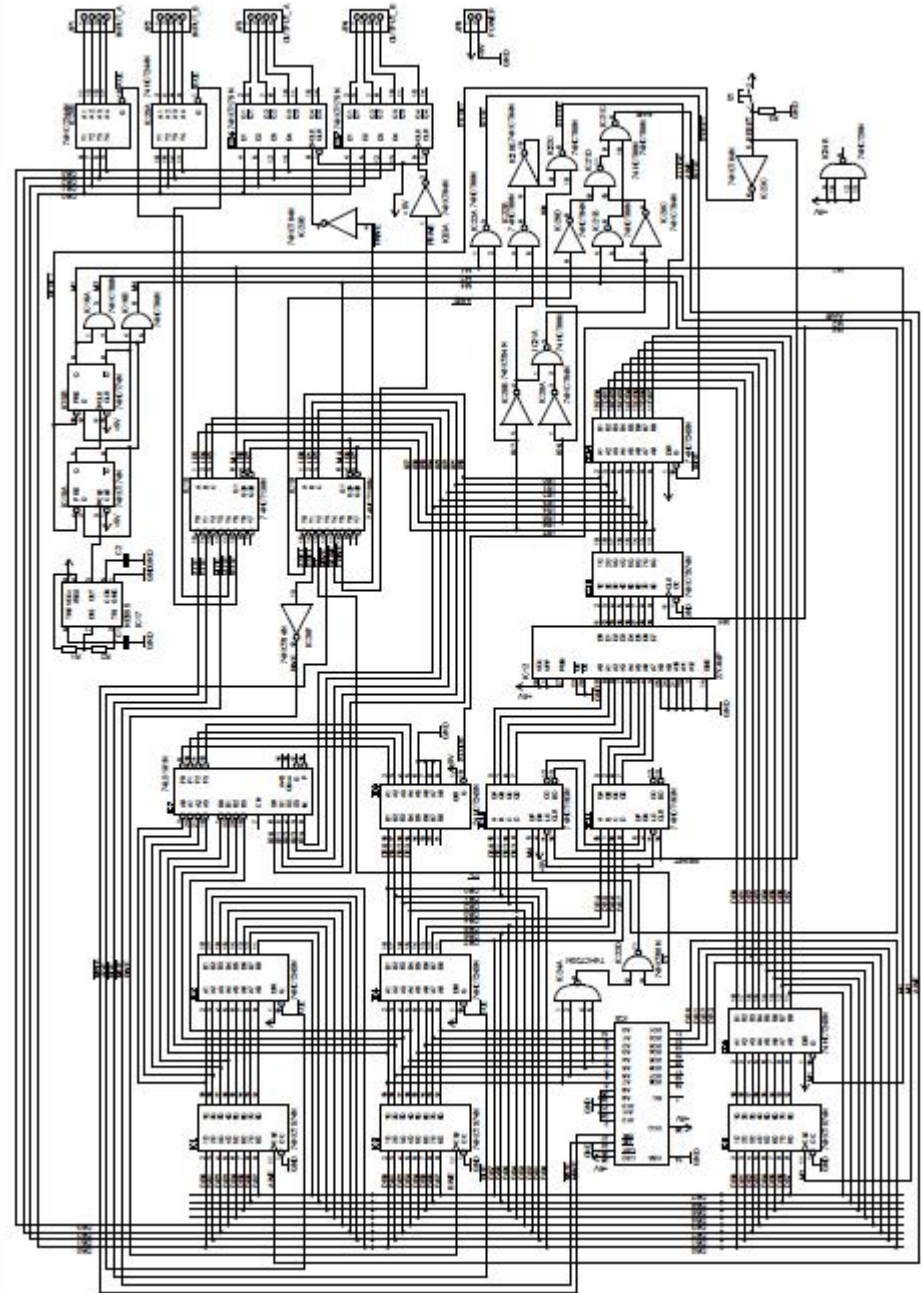
课后作业

• 4) Project

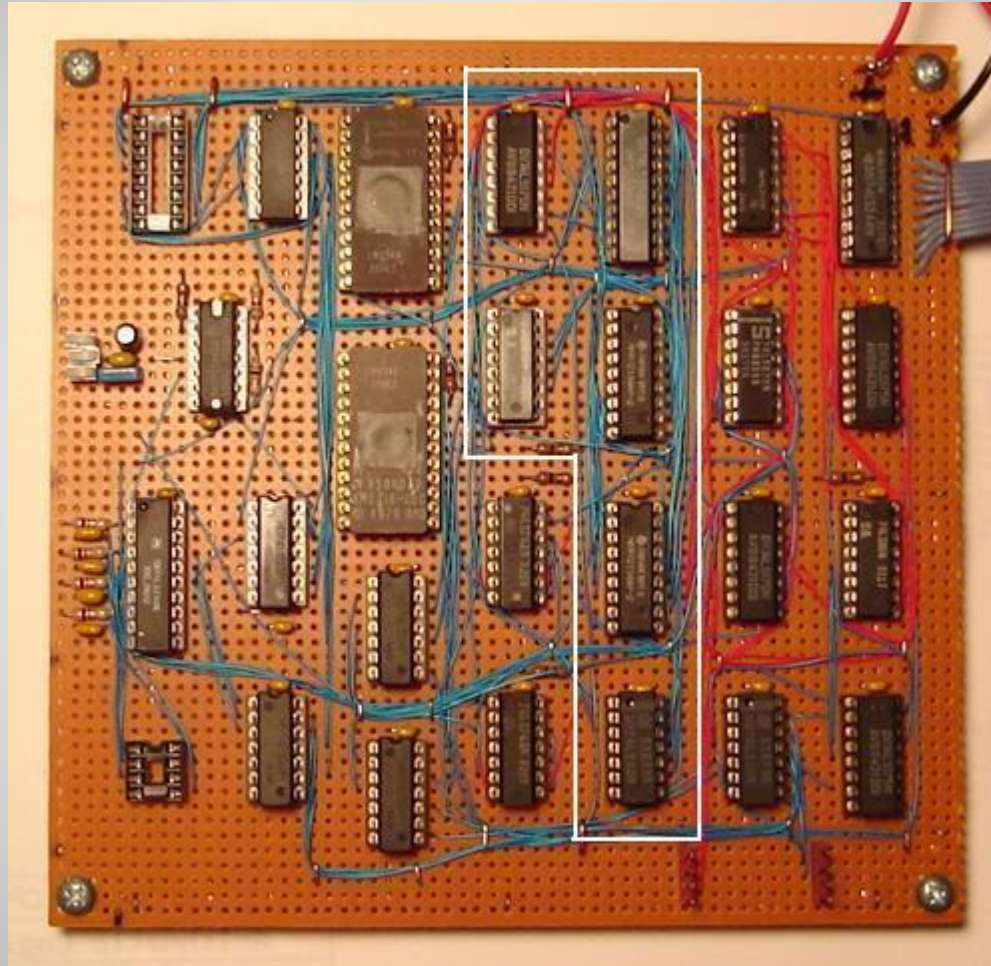
- **Project1（必做）**：将讲义P31页图8.33的框图用具体型号的数字集成电路设计实现为具体电路图。（**交报告和电路图**）
- **Project2（选做平时成绩加分）**：用具体逻辑电路描述设计并FPGA实现一个4bit简单CPU实物。（**期末停课一周前完成**）

4bit Simple CPU

- ALU
- 时钟
- PC计数器
- 寄存器
- 程序存储器
- 寄存器组
- 总线



4bit Simple CPU



参考资料

- www.ttlcpu.com